MODULE 5

DEEP LEARNING AND MACHINE LEARNING MASTERY IN VISION AND TRANSFER LEARNING

MLFP — ML Foundations for Professionals — Terrene Open Academy

"Every major DL architecture. One paradigm per lesson. All implemented."

WHAT YOU WILL LEARN

BY THE END OF MODULE 5

Build autoencoders — vanilla, denoising, VAE, convolutional
Implement CNNs with ResNet, SE blocks, mixed precision
Train LSTMs and GRUs with attention mechanisms
Derive self-attention from scratch, fine-tune transformers
Build GANs (DCGAN, WGAN) and understand diffusion
Apply GNNs to graph-structured data
Transfer pre-trained models to new CV and NLP tasks
Implement RL — DQN, PPO, SAC for business applications

THREE DEPTH LAYERS

FOUNDATIONS — Everyone follows. Intuition, analogies, visual diagrams.

THEORY — Full derivations, gate equations, loss functions.

ADVANCED — Paper references, architectural variants, edge cases.

A banker and a PhD sit in the same classroom. Both leave having learned something new.

YOUR JOURNEY: 8 LESSONS

#	Lesson	Paradigm	You Will Be Able To...
5.1	Autoencoders	Unsupervised DL	Build VAEs and generate new data
5.2	CNNs & Computer Vision	Spatial Features	Train ResNets with modern enhancements
5.3	RNNs & Sequence Models	Temporal Features	Implement LSTMs with attention
5.4	Transformers	Parallel Attention	Derive self-attention, fine-tune BERT
5.5	GANs & Diffusion	Generative Models	Build DCGAN and WGAN
5.6	Graph Neural Networks	Graph Features	Build GCNs for node classification
5.7	Transfer Learning	Knowledge Transfer	Fine-tune and deploy with ONNX
5.8	Reinforcement Learning	Interaction Learning	Implement DQN and PPO

KAILASH ENGINES IN MODULE 5

ONNXBRIDGE

Export any trained model to ONNX format for cross-platform deployment. Used in Lessons 5.2 and 5.7.

INFERENCESERVER

`predict`, `predict_batch`, `warm_cache` — serve production predictions. Used in Lesson 5.7.

MODELVISUALIZER

Visualise training curves, latent spaces, attention maps, and generated outputs across all lessons.

RLTRAINER

Train RL agents with DQN, PPO, SAC. Custom Gymnasium environments. Used in Lesson 5.8.

DL TOOLKIT REFRESHER (FROM M4)

WHAT YOU ALREADY KNOW

Forward pass: inputs flow through layers to produce output

Backpropagation: gradients flow backward to update weights

Gradient descent: SGD, Adam, learning rate scheduling

Dropout: randomly zero activations to prevent overfitting

Batch normalisation: normalise layer inputs for stable training

Quick exercise: Build a 2-layer classifier. Good — now we modify this architecture for UNSUPERVISED learning. That is an autoencoder.

The shift: M4 trained networks to predict labels (supervised). M5 trains networks to learn *representations* — compressed, generated, attended, transferred.

DL DIAGNOSTICS TOOLKIT

THE DOCTOR'S BAG

Five instruments you carry into every model you train

"The architecture isn't what separates good ML engineers from great ones. The diagnostic instinct is."

Turn to your neighbour (2 min): name a training run that failed on you — and what you did about it. *(Self-study: jot it in your notebook.)*

THE FIVE INSTRUMENTS

Stethoscope

Loss curves

asks: "is it learning?"

Blood Test

Gradient flow

asks: "are signals flowing?"

X-Ray

Activation maps

asks: "what did it learn?"

Vital Signs

Dashboard

asks: "what's the trend?"

Prescription

Interventions

asks: "what do I chang

DIAGNOSE — what's wrong?

TREAT

First four tell you what's wrong. Fifth tells you what to do about it.

MEET `DLDiagnostics` — YOUR TOOLKIT IN ONE OBJECT

Every slide in this toolkit section has a corresponding method. You never write raw hooks.

```
from shared.mlfp05.diagnostics import DLDiagnostics

# ── Setup (one line) ──────────────────────────────────────
diag = (DLDiagnostics(model)
        .track_gradients()
        .track_activations()
        .track_dead_neurons())

# ── Training (two lines per batch) ────────────────────────
for batch in dataloader:
    loss = train_step(...); loss.backward(); optimizer.step()
    diag.record_batch(loss=loss.item(),
                      lr=optimizer.param_groups[0]["lr"])
diag.record_epoch(val_loss=val)

# ── Diagnose (one line each) ──────────────────────────────
diag.plot_training_dashboard().show()   # all 5 instruments, one figure
diag.report()                           # auto-diagnosis text
diag.grad_cam(image, target_class)      # for CNN exercises (Lesson 5.2)
DLDiagnostics.lr_range_test(model, dataloader)   # pre-training LR probe (covered in 5.2 / 5.7)
```

Why one object? Hooks, tensors, and plot logic are fiddly to get right — and wrong hooks silently break your diagnosis. `DLDiagnostics` owns the plumbing. You name what to track, call `record_batch` / `record_epoch`, and ask for plots.

PREREQUISITE — WHAT IS LOSS?

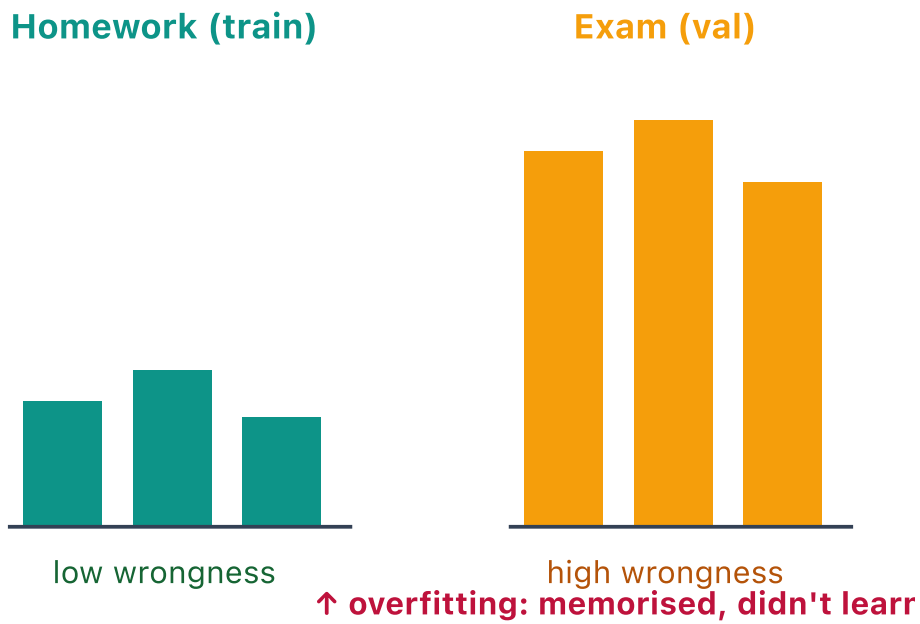
Loss = how wrong the model is, on average

Train loss = wrongness on examples it has *seen*

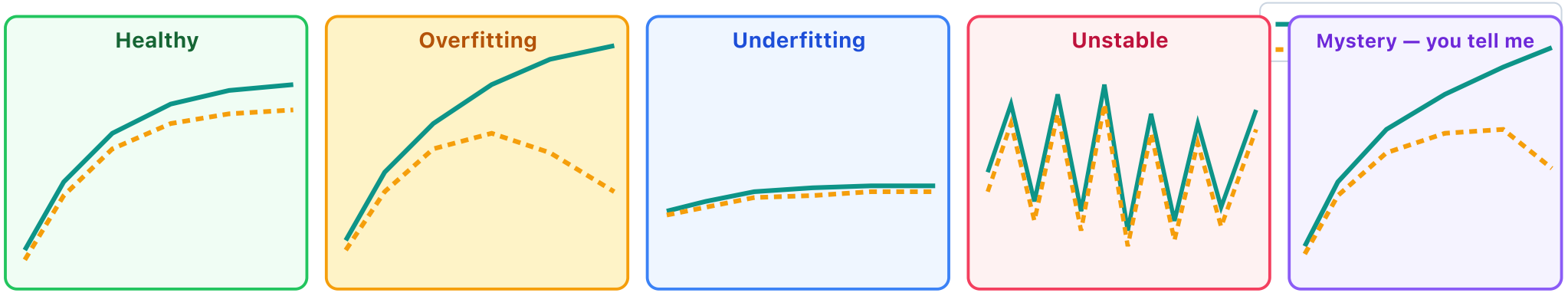
Val loss = wrongness on examples it has *NOT seen* — the held-out exam

A student who aces homework but bombs the exam = **overfitting**

> **Reading rule:** judge the model by the *gap* between homework and exam, not homework alone.



INSTRUMENT 1: LOSS CURVES — THE STETHOSCOPE



Healthy	Overfitting	Underfitting	Unstable	Mystery — you tell me
Both drop, converge, gap stable	Train drops, val climbs back up	Both plateau, loss remains high	Oscillations, spikes, divergence	name this shape
→ continue training	→ early stop / regularise	→ more capacity / train longer	→ lower LR / clip gradients	→ vote before reveal

vote: healthy / overfit / underfit / unstable

What the stethoscope tells you:

The SHAPE of the loss curve is the single most compressed signal about training health.
Look at it every epoch. Name the shape. Act on the shape.

First use: **Exercise 5.1.1** (autoencoder training), this afternoon.

READING LOSS CURVES — IN PRACTICE

THREE HABITS THAT SAVE HOURS

Train + val on ONE plot — the gap is the diagnosis, not each curve alone

Smooth with a moving average (window ~ 10 steps) — noisy batch loss hides the trend

Log-scale the y-axis early — the first 100 steps compress 90% of the learning

> **Rule:** If you cannot see the gap between train and val on your plot, your plot is wrong. Re-plot before you re-train.

```python
import plotly.graph_objects as go
import numpy as np

def plot_losses(train, val, smooth=10):
    # moving average smooths per-batch noise
    kernel = np.ones(smooth) / smooth
    tr = np.convolve(train, kernel, mode="valid")
    vl = np.convolve(val,   kernel, mode="valid")

    fig = go.Figure()
    fig.add_trace(go.Scatter(y=tr, name="train", line=dict(color="#0D9488")))
    fig.add_trace(go.Scatter(y=vl, name="val",   line=dict(color="#F59E0B", dash="dash")))
    fig.update_yaxes(type="log")  # log-scale catches early learning
    fig.update_layout(
        title="Loss curves (smoothed, log-y)",
        xaxis_title="step", yaxis_title="loss",
    )
    return fig

# Use it every single training run
fig = plot_losses(history["train"], history["val"])
fig.write_html("loss_curves.html")
```

First use: **Exercise 5.1.1** (autoencoder training), this afternoon.

PREREQUISITE — WHAT IS A GRADIENT?

Analogy: imagine your model is a room with 10,000 thermostats. Each controls a tiny part of the temperature (the model's output). The *gradient* tells each thermostat which way to nudge (hotter / colder) to bring the room closer to your target temperature.

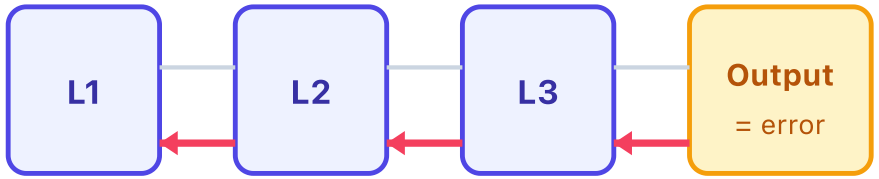
Gradient = how much each knob (weight) in the model needs to turn to reduce error

Backprop = the algorithm that computes those turn-amounts layer by layer, working *backward* from the output

Gradient ≈ 0 for a layer → its knobs aren't turning → **it isn't learning**

Gradient is huge → knobs spinning wildly → **model destabilises**

Reading rule: the gradient at a layer is a stethoscope for that layer — it tells you whether the layer is even participating in learning.



gradient flows backward — layer by layer

error propagates back; each L gets a “turn this much” signal

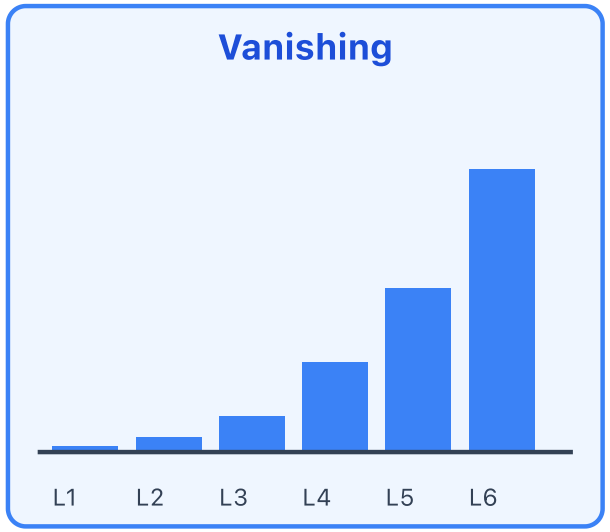
forward = predict • backward = learn

THEORY

INSTRUMENT 2: GRADIENT FLOW — THE BLOOD TEST

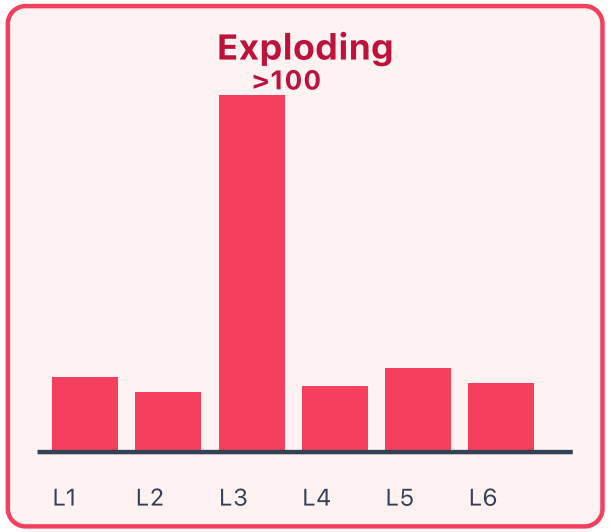
RMS 1e-5 – 1e-2 across all layers

signal reaches every layer



early-layer RMS < 1e-5 (or update ratio < 1e-4)

input-side layers learn nothing



one layer RMS > 1e-2 or update ratio > 0.1

updates overshoot, loss diverges

First use: **Exercise 5.3** (RNN vanishing-gradient detection).

GRADIENT FLOW — CODE & THRESHOLDS

```python
from shared.mlfp05.diagnostics import DLDiagnostics

diag = DLDiagnostics(model).track_gradients()

for batch in dataloader:
    loss = train_step(model, batch)
    loss.backward()
    optimizer.step()
    diag.record_batch(
        loss=loss.item(),
        lr=optimizer.param_groups[0]["lr"],
    )

diag.plot_gradient_flow().show()
diag.report()   # auto-diagnoses vanishing / exploding
```

▶ Under the hood (5-line hook reference)

THRESHOLDS WORTH MEMORISING

Per-element RMS = $\|\nabla\| / \sqrt{\text{numel}}$ • update ratio = $\|\nabla W\| / \|W\|$

Signal	Verdict
RMS < 1e-5 *or* ratio < 1e-4	**vanishing**
RMS 1e-5 – 1e-2	healthy
RMS > 1e-2 *or* ratio > 0.1	**exploding**
NaN	**dead**

Why not raw gradient norm? Gradient norm scales with tensor size. A 1024×1024 weight's grad norm can be 1000× larger than a 32×32 for the same per-element magnitude. Per-element RMS and update ratio are scale-invariant — what LLM training dashboards actually show.

Modern (2025): ZClip adapts the clip threshold to the running distribution of gradient norms — 2–4x fewer loss spikes on long-context LLM training than fixed `clip_grad_norm_(max_norm=1.0)`.

First use: **Exercise 5.3** (RNN vanishing-gradient detection).

INSTRUMENT 3A: STATISTICAL X-RAY (ACTIVATION X-RAY — PART 1 OF 2)

Runs inside the training loop — one number per layer, every N steps.

WHAT TO LOG PER LAYER

mean — drifting toward 0 or ±∞?

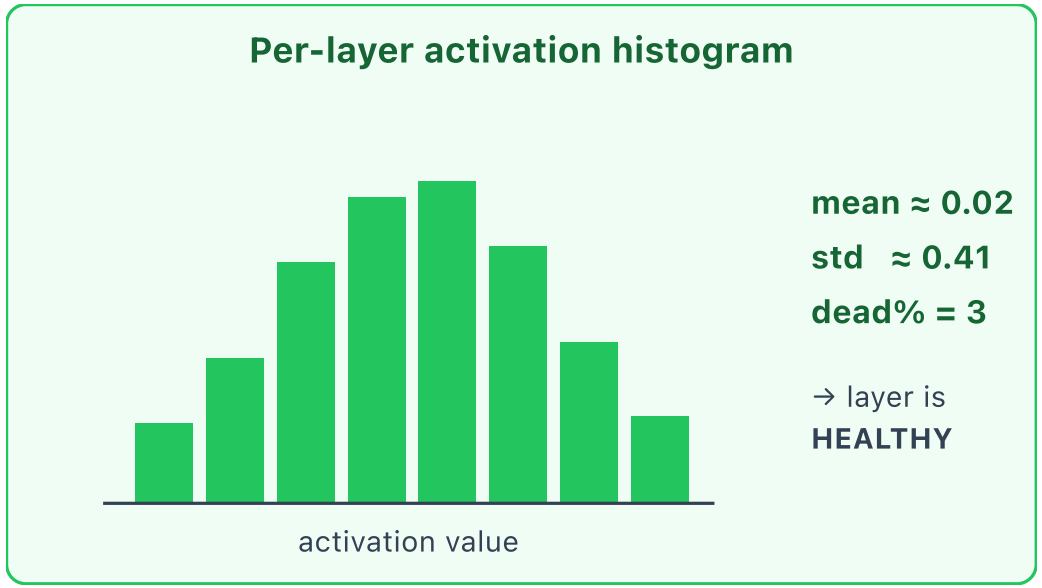
std — collapsing (dead) or exploding?

dead fraction — % of units at exactly 0

saturation — % at ±1 (tanh) or 0/1 (sigmoid)

One-line diagnostic: `(act == 0).float().mean()`

> **Healthy pattern:** mean ≈ 0, std in [0.3, 1.0], dead% < 10%. If any layer drifts, you see it within 100 steps.



First use: **Exercise 5.2.1** (CNN baseline health check).

INSTRUMENT 3B: ATTRIBUTION X-RAY (ACTIVATION X-RAY — PART 2 OF 2)

Runs at inference-time — answers "which input drove this prediction?"

THREE METHODS, ONE QUESTION

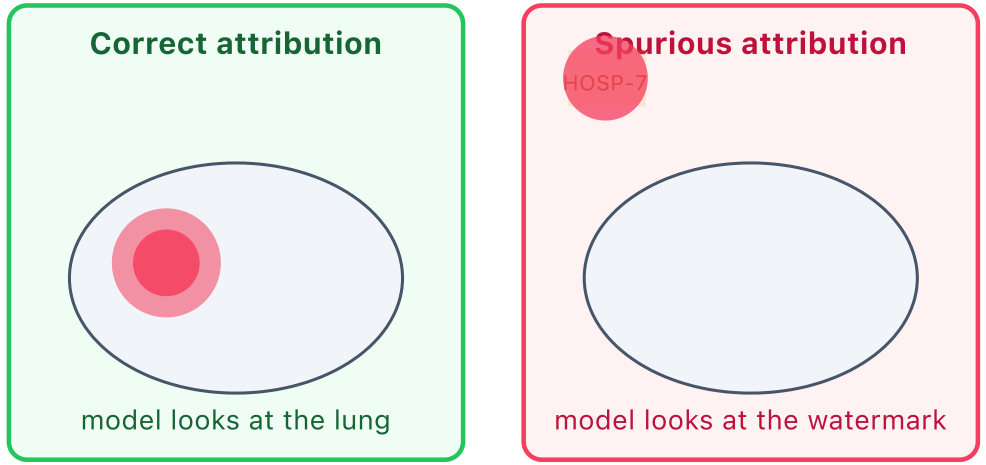
Grad-CAM (CNNs) — heatmap over the image

Attention rollout (transformers) — token-to-token weight map

Integrated gradients (any model, any input)

Healthcare — Zech et al., 2018 (*Nature Medicine*): a chest-X-ray classifier achieved 95%+ accuracy. Grad-CAM revealed it was looking at the hospital's *image watermark*, not the lung. Accuracy held on the same hospital, *collapsed* on transfer. Attribution caught a model that every other metric passed.

Finance — the postcode proxy: a credit-risk model hits 89% AUC. SHAP/attribution reveals it uses *postcode* (a proxy for race/income) instead of the income feature you put in the brief. Legal says no, model says yes. Attribution is what forced the conversation.



Same accuracy — same hospital.

Collapses on transfer. Attribution caught it.

First use: **Exercise 5.2.2** (CNN Grad-CAM interpretability).

DEAD NEURONS — DETECT & FIX

Why dead ReLUs happen: a large negative bias shove pushes every input to a unit below 0 → ReLU outputs 0 → gradient is 0 → the unit cannot recover. Silent — loss still decreases because the live units absorb the work.

DETECTING

```python
from shared.mlfp05.diagnostics import DLDiagnostics

diag = DLDiagnostics(model).track_dead_neurons()

# run 1 warm-up epoch of training here...

diag.plot_dead_neurons().show()
# Auto-warning in diag.report():
#   "conv3 has 67% dead neurons — consider GELU"
```

Rule of thumb: any layer > 50% dead is a problem. The library flags it for you.

▶ Under the hood (forward hooks)

FIXING

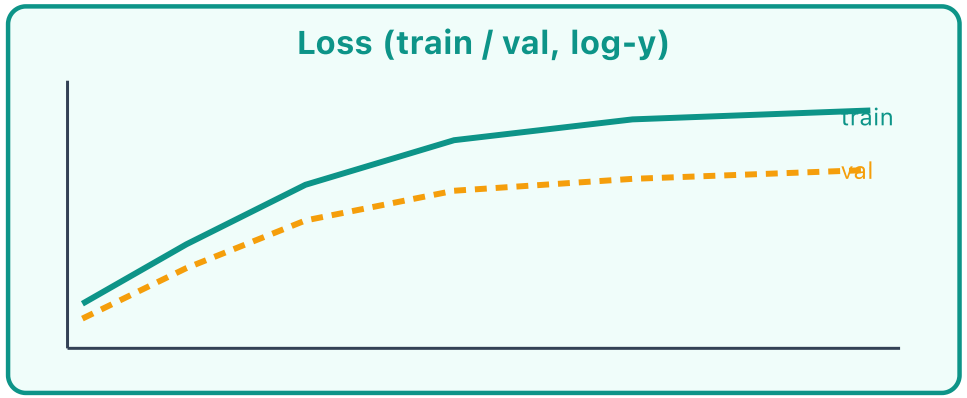
Step 1: swap the activation.
`ReLU → GELU` (transformers),
`ReLU → LeakyReLU(0.01)` (CNNs),
`ReLU → SiLU/Swish` (modern).

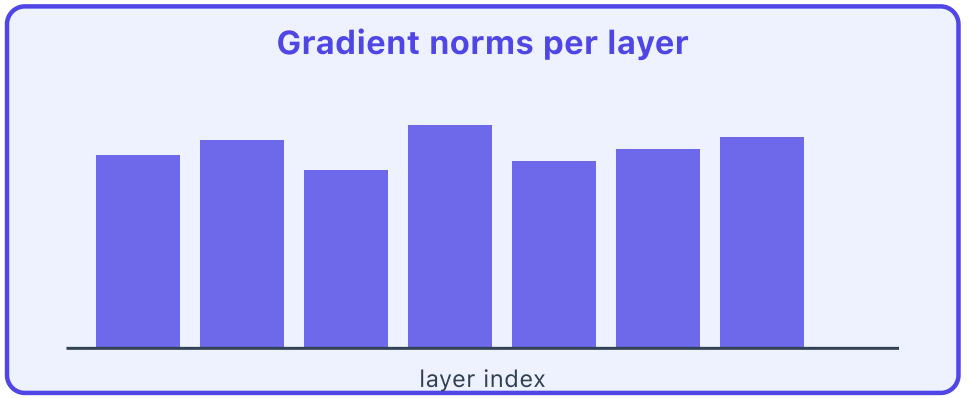
Step 2: re-initialise with Kaiming.
`nn.init.kaiming_normal_(w, nonlinearity='relu')`
Scales variance so ReLU's 50% zeroing does not collapse signal depth.

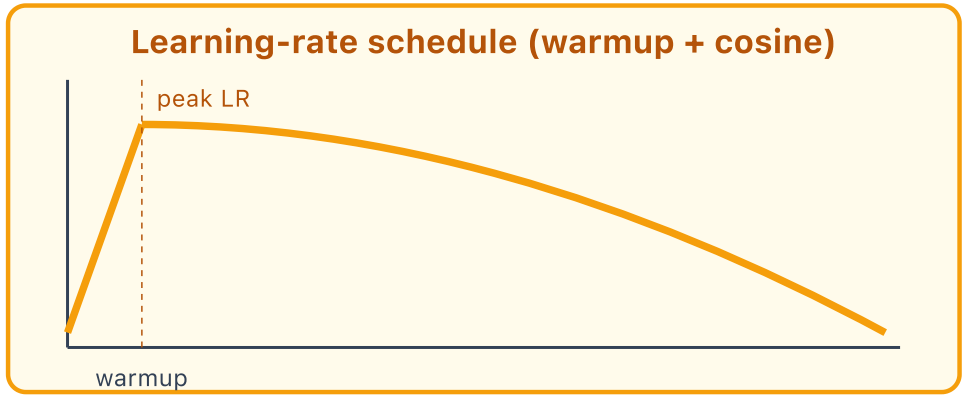
First use: **Exercise 5.2.1** (CNN baseline health check).

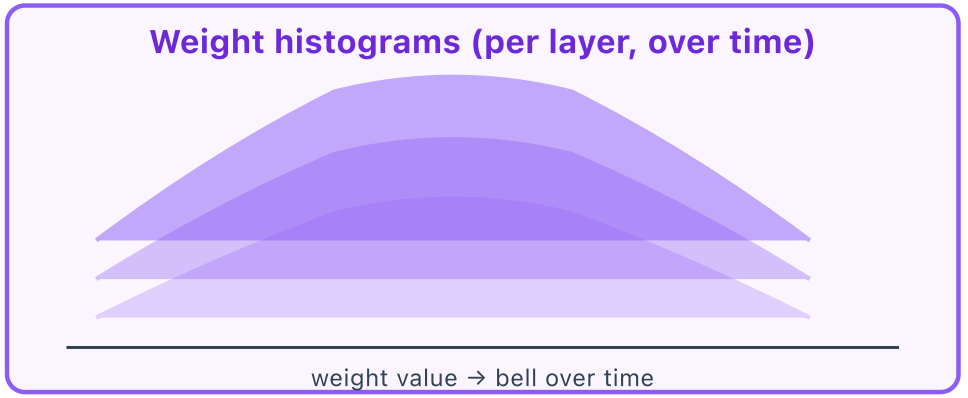
INSTRUMENT 4: TRAINING DASHBOARD — *THE PATIENT CHART*

The integration — all four previous readings on one screen.









Used in **every lesson** of M5.

INSTRUMENT 5: THE PRESCRIPTION PAD

SYMPTOM	PRESCRIPTION
Loss oscillating	**reduce LR by 3–10x** or add linear warmup (500–2000 steps)
Val loss diverges while train drops	**add regularisation** dropout / weight decay / early-stop / augment
Gradients vanishing (RMS < 1e-5 or update ratio < 1e-4)	**add skip connections / GELU** or reduce depth; check for Sigmoid/Tanh blocks
Gradients exploding (RMS > 1e-2 or update ratio > 0.1, or NaN)	**clip (ZClip) / reduce LR / check init** Kaiming for ReLU; Xavier for Tanh
Dead neurons > 50%	**GELU / LeakyReLU + Kaiming init** swap activation THEN re-initialise weights
Loss not decreasing at all	**increase capacity OR check data pipeline** overfit to 1 batch first; if that fails, pipeline is broken
Sudden loss spikes mid-training	**adaptive clipping + bad-batch audit** log the offending batch; check for label noise / outliers

Glossary (for this prescription pad):
GELU / LeakyReLU: smoother ReLU variants that don't kill neurons. • **Kaiming init**: weight starting values that preserve signal through ReLU layers. • **Skip connection**: a shortcut wire that lets gradient bypass a layer. • **Warmup**: start with a tiny LR, ramp up over 500–2000 steps. • **ZClip**: adaptive gradient clipping that tracks the running gradient distribution.

Reference **throughout the module**.

HOW THE TOOLKIT THREADS THROUGH M5

Lesson	Topic	Instruments used	What you will diagnose
5.1	Autoencoders	Loss, activations	Posterior collapse in VAE (KL → 0); reconstruction-vs-KL trade-off
5.2	CNNs & Vision	All five	LR range test, Grad-CAM attribution, dead ReLUs in early conv blocks
5.3	RNNs & LSTMs	Gradients, dashboard	Vanishing gradients through time; BPTT stability; clipping threshold
5.4	Transformers	All five + attention maps	Warmup length, attention rollout, RMSNorm vs LayerNorm ablation
5.5	GANs & Diffusion	Loss, gradients, prescription	Mode collapse, generator/discriminator balance, WGAN gradient penalty
5.6	Graph Neural Nets	Gradients, activations	Over-smoothing at depth > 4 layers; dead node embeddings
5.7	Transfer Learning	Loss, gradients	Catastrophic forgetting; discriminative LRs; layer-wise unfreeze schedule
5.8	Reinforcement Learning	Dashboard (reward curves) + prescription	Reward hacking, policy entropy collapse, value-loss divergence

Every lesson's first exercise begins with: "Run the diagnostic protocol. Report the five instrument readings. Then train."

YOUR DIAGNOSTIC WORKFLOW

THE PROTOCOL — EVERY MODEL, EVERY RUN

Four checks, then a decision.

Train 1 epoch (or 500 steps) as a smoke test

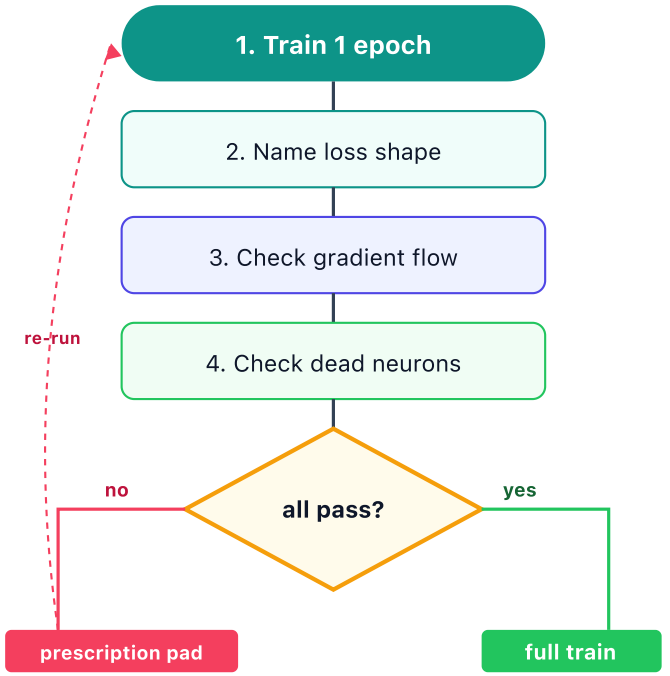
Check loss curves — name the shape (healthy / over / under / unstable)

Check gradient flow — vanishing (RMS < 1e-5) or exploding (RMS > 1e-2)?

Check dead neurons — any activation layer > 50% dead?

Decision: all pass → full train. Any fails → prescription pad, apply ONE fix, repeat.

> **Cardinal rule:** change ONE thing at a time. If you change LR, activation, and init together, you cannot tell which fix worked.



APPENDIX: MODERN PRACTICE NOTES

Tactical knowledge — reference as you build

Modern architecture choices, the LR range test, and scaling laws. Skim now; return when you need them.

ADVANCED

MODERN ARCHITECTURE CHOICES (2026)

Component	Legacy default (pre-2020)	Current best practice (2026)	Why the upgrade
Activation	ReLU	SwiGLU (transformers), GELU (general)	No dead zone; smooth gradient; +0.5–1.0 pt on Llama-class benchmarks
Normalisation	BatchNorm	RMSNorm (transformers), LayerNorm (elsewhere)	RMSNorm drops mean-subtract step: ~5–10% faster unfused, more with kernel fusion; same quality (Llama 3)
Optimizer	Adam with L2 in loss	AdamW (decoupled weight decay)	L2-in-loss interacts badly with Adam's adaptive LR; AdamW fixes it (Loshchilov 2019)
LR schedule	Step decay / ReduceLROnPlateau	Linear warmup + cosine decay	Warmup prevents early explosion; cosine gives smooth annealing without tuning steps
Grad clipping	Fixed `max_norm=1.0`	ZClip (adaptive, 2025)	Adapts to the running gradient distribution; 2–4x fewer loss spikes on long runs
Initialisation	Xavier (uniform)	Kaiming for ReLU-family, Xavier for Tanh/Sigmoid	Kaiming preserves signal variance through ReLU's 50% zeroing
Hyperparameter transfer	Tune LR / init / etc. separately at every scale	µP (Maximal Update Parametrization)	Tune at 100M, transfer zero-shot to 7B+ (Yang & Hu, 2021)
Precision	FP32 throughout	BF16 forward + FP32 optimiser states (mixed precision)	2–4x memory / throughput; BF16 dynamic range avoids FP16 NaN issues

Rule: If you copy an architecture from a paper published before 2022, replace every row above with the 2026 column before you start training.

THE LR RANGE TEST (LESLIE SMITH, 2017)

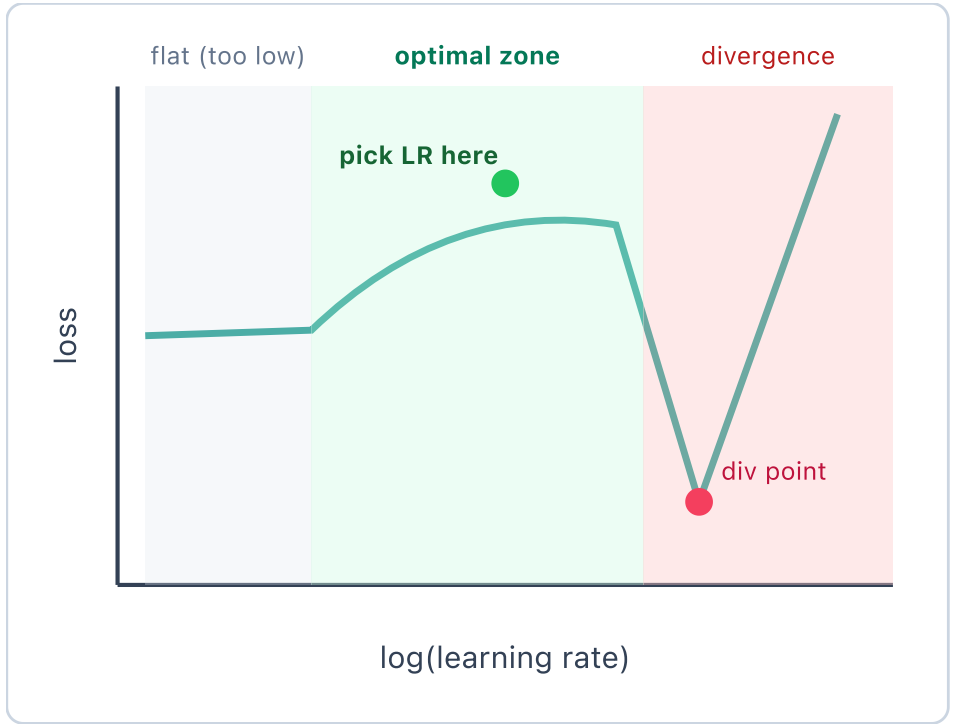
Before you train, find your learning rate. Do NOT guess.

- Train for ~100–500 steps, increasing LR exponentially from 1e-8 to 10
- Plot loss vs log(LR)
- Find the divergence point (loss shoots up)
- Pick training LR = **1/3 to 1/10** of the divergence LR

```
from shared.mlfp05.diagnostics import DLDiagnostics

result = DLDiagnostics.lr_range_test(
    model, dataloader,
    lr_min=1e-7, lr_max=10, steps=200,
)
print(f"Use lr = {result['suggested_lr']:.1e}")
#   → rule of thumb: divergence-point LR / 10
result["figure"].show()     # the curve below
```

Three lines replace a custom scheduler. The helper handles LR ramp, divergence detection, and plotting.

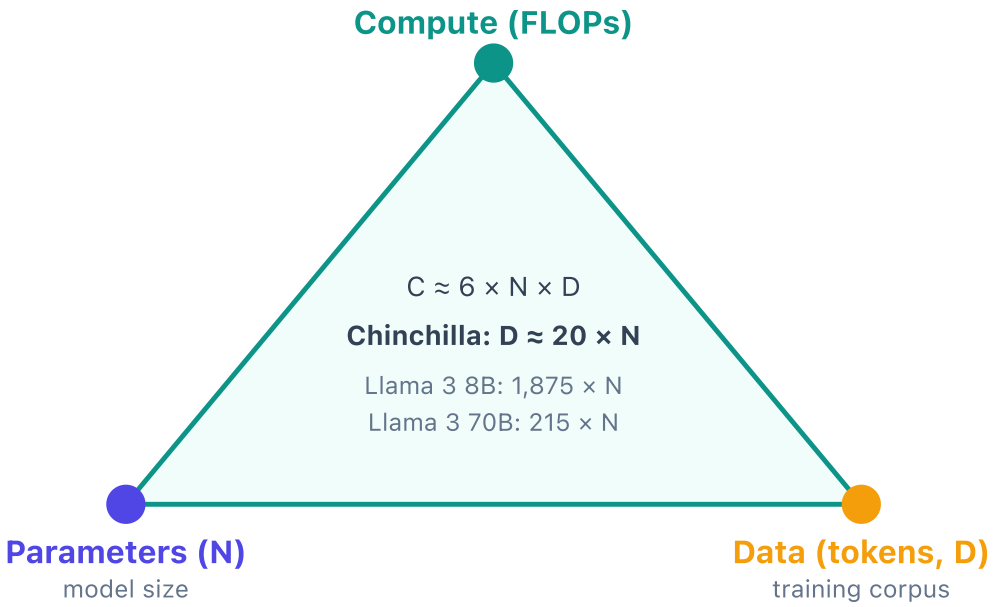


Used by fast.ai, timm, HuggingFace Trainer.

ADVANCED

SCALING LAWS — HOW MUCH TRAINING?

THE COMPUTE = PARAMS × DATA TRIANGLE



Read: pick any two corners; the third follows.

HOW TO READ IT

Fixed compute budget: you choose *how to spend* it — bigger model with less data, or smaller model with more data?

Chinchilla (20 tokens/param): balanced — the cheapest way to *train* a model to a given quality.

Llama 3 8B (1,875 tokens/param): deliberately over-trained. *Why?* Training is one-time; inference is forever. A smaller model that hit the same quality is cheaper to *deploy*. Shrink N, overtrain D.

Decision rule: small + overtrained = cheaper to deploy. Big + Chinchilla-optimal = cheaper to train. Pick by where the cost lives.

LESSON 5.1

AUTOENCODERS

Unsupervised DL — Learning Compressed Representations

"What if the network had to describe itself in fewer words than it has?"

THE AUTOENCODER ARCHITECTURE

THREE COMPONENTS

Encoder — compresses input to latent representation

Latent Space — compressed bottleneck (fewer dimensions than input)

Decoder — reconstructs input from latent representation

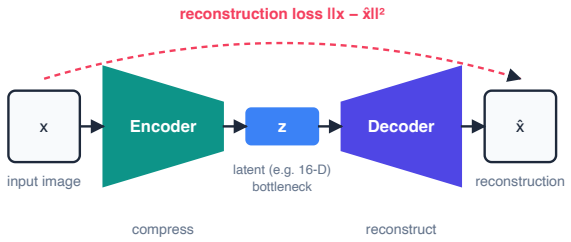
Analogy: Summarise a book in one sentence (encoder). Someone else tries to rewrite the book from your sentence (decoder). The better the summary, the closer the rewrite.

WHY "AUTO"?

The target IS the input. No labels needed. The network learns to reconstruct itself through a bottleneck.

$$\text{Input } x \xrightarrow{\text{Encoder}} z \xrightarrow{\text{Decoder}} \hat{x} \approx x$$

The bottleneck forces the network to learn only the essential features — it cannot memorise.



RECONSTRUCTION LOSS

$$\mathcal{L}_{\text{recon}} = \|x - \text{decoder}(\text{encoder}(x))\|^2$$

Mean Squared Error between input and reconstruction

WHAT THIS MEASURES

- Pixel-by-pixel difference for images
- Feature-by-feature difference for tabular data
- Lower loss = better reconstruction = better learned features

ALTERNATIVE LOSSES

- **Binary cross-entropy** — for binary/normalised inputs
- **Perceptual loss** — compare in feature space, not pixel space
- **SSIM loss** — structural similarity for images

VARIANT 1: VANILLA AUTOENCODER

ARCHITECTURE

Fully connected layers only

Undercomplete: latent dim < input dim

Simplest form — the baseline

```
class VanillaAutoencoder(nn.Module):
    def __init__(self, input_dim, latent_dim):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 128),
            nn.ReLU(),
            nn.Linear(128, latent_dim))
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 128),
            nn.ReLU(),
            nn.Linear(128, input_dim),
            nn.Sigmoid())
```

WHEN TO USE

Dimensionality reduction (like PCA, but nonlinear)

Feature extraction for downstream tasks

Anomaly detection (high reconstruction error = anomaly)

Key insight: If latent_dim = input_dim, the network just copies. The bottleneck is what forces learning.

VARIANT 2: DENOISING AUTOENCODER (DAE)

CORE IDEA

Corrupt the input with noise

Train the network to reconstruct the *clean* version

Forces learning of robust features, not surface patterns

$$\mathcal{L}_{\text{DAE}} = \|x - \text{decoder}(\text{encoder}(\tilde{x}))\|^2$$

where $\tilde{x} = x + \epsilon$, noise $\epsilon \sim \mathcal{N}(0, \sigma^2)$

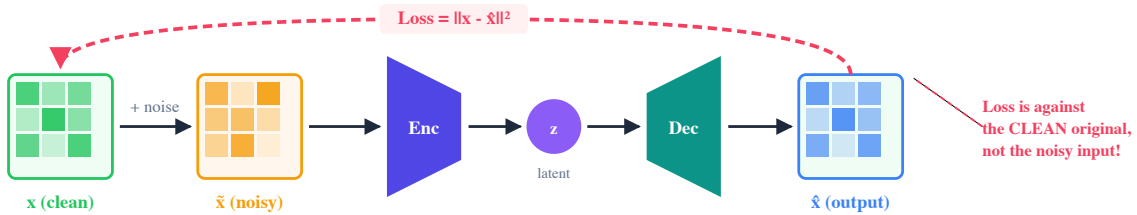
NOISE TYPES

Gaussian noise: add random values from normal distribution

Masking noise: randomly zero out input features

Salt-and-pepper: random pixels set to 0 or 1

Why it works: The network cannot memorise noisy inputs. It must learn the underlying data distribution to denoise. This produces more generalisable features than the vanilla variant.



VARIANT 3: VARIATIONAL AUTOENCODER (VAE)

THE KEY INNOVATION

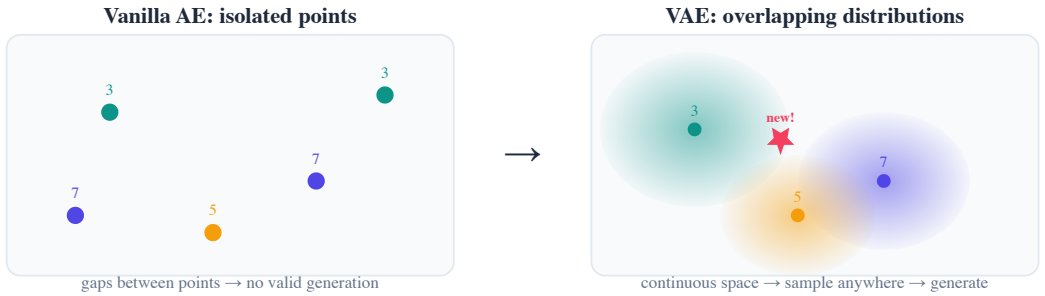
Instead of encoding to a fixed point, the VAE encodes to a **probability distribution** over latent space.

- Encoder outputs μ and σ (mean and std dev)
- Sample from $\mathcal{N}(\mu, \sigma^2)$ to get latent code
- Can **generate new data** by sampling from latent space

WHY DISTRIBUTIONS?

Analogy: Vanilla AE says "this digit lives at coordinate (3.2, -1.7)." VAE says "this digit lives *somewhere around* (3.2, -1.7) with some uncertainty." The fuzziness means nearby points also produce valid digits — enabling generation.

Formally: The encoder approximates the posterior $q_\phi(z|x) \approx p(z|x)$. The decoder models the likelihood $p_\theta(x|z)$.



VAE LOSS: THE ELBO

$$\mathcal{L}_{\text{VAE}} = \underbrace{\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)]}_{\text{Reconstruction}} - \underbrace{D_{\text{KL}}(q_\phi(z|x)\|p(z))}_{\text{Regularisation}}$$

Evidence Lower Bound (ELBO) — maximise this

TERM 1: RECONSTRUCTION

How well does the decoder reconstruct the input from the sampled latent code? Higher is better.

TERM 2: KL DIVERGENCE

How close is the learned latent distribution to the prior $\mathcal{N}(0, I)$? Acts as regulariser, preventing the latent space from collapsing to isolated points.

KL closed-form (Gaussian): $D_{\text{KL}} = -\frac{1}{2}\sum_{j=1}^{J}(1 + \log\sigma_j^2 - \mu_j^2 - \sigma_j^2)$ — no sampling needed for this term.

THE REPARAMETERISATION TRICK

THE PROBLEM

Sampling $z \sim \mathcal{N}(\mu, \sigma^2)$ is a **stochastic operation**. Gradients cannot flow through random sampling. Backprop breaks.

THE SOLUTION

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

Move randomness to an input — gradients flow through μ and σ

STEP 1

Encoder outputs μ and $\log \sigma^2$ (log-variance for numerical stability)

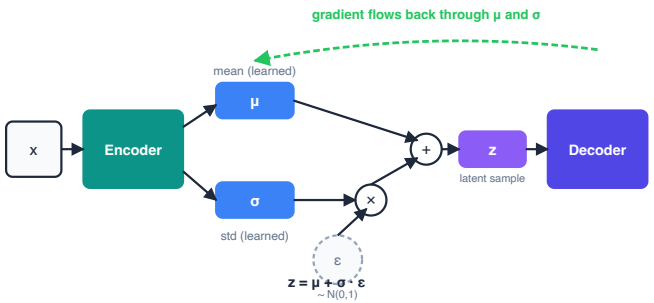
STEP 2

Sample ϵ from standard normal (external randomness)

STEP 3

Compute $z = \mu + \exp(0.5 \cdot \log \sigma^2) \cdot \epsilon$

Why it works: ϵ is treated as a constant input. The network learns μ and σ deterministically. Gradients flow through both.



VARIANT 4: CONVOLUTIONAL AUTOENCODER

WHY CONVOLUTIONS?

Fully connected layers ignore spatial structure

Conv layers preserve spatial relationships

Far fewer parameters for image data

ARCHITECTURE

Encoder: Conv2d + ReLU + MaxPool (downsample)

Decoder: ConvTranspose2d + ReLU (upsample)

Mirror the encoder structure in reverse

```
class ConvAutoencoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Conv2d(1, 16, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),
            nn.Conv2d(16, 4, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2))
        self.decoder = nn.Sequential(
            nn.ConvTranspose2d(4, 16, 2,
                               stride=2),
            nn.ReLU(),
            nn.ConvTranspose2d(16, 1, 2,
                               stride=2),
            nn.Sigmoid())
```

ADVANCED

ADDITIONAL AUTOENCODER VARIANTS (SURVEY)

Variant	Key Idea	Use Case
Sparse AE	L1 penalty on activations — most neurons stay inactive	Feature selection, interpretability
Contractive AE	Penalty on Jacobian of encoder — small input changes, small latent changes	Robust representations
Stacked AE	Train layer-by-layer, then fine-tune end-to-end	Very deep encoders
Recurrent AE	LSTM/GRU encoder-decoder for sequences	Time series, text (Lesson 5.3)
CVAE	Conditional VAE — condition generation on class labels	Controlled generation

Research note: VQ-VAE (Vector Quantised) discretises the latent space. Used in DALL-E 1 for image generation. Bridges autoencoders and modern generative AI.

EXERCISE 5.1: AUTOENCODER WORKSHOP

TASKS

- Implement vanilla autoencoder on MNIST (latent dim=32)
- Implement denoising autoencoder — compare reconstruction quality
- Implement VAE with reparameterisation trick
- Generate new digits by sampling from VAE latent space
- Implement convolutional autoencoder — compare with FC variants

ASSESSMENT CRITERIA

- All 4 variants implemented and trained
- VAE generates plausible new images
- Latent space visualisation shows meaningful structure (t-SNE or PCA)
- Reconstruction quality compared across variants

> **Stretch goal:** Interpolate between two digits in VAE latent space. Watch the smooth transition.

LESSON 5.1 SUMMARY

Autoencoders learn compressed representations by reconstructing their own input through a bottleneck.

KEY EQUATIONS

Reconstruction: $\mathcal{L} = \|x - \hat{x}\|^2$

ELBO: $\mathbb{E}[\log p(x|z)] - D_{\mathrm{KL}}$

Reparam: $z = \mu + \sigma \odot \epsilon$

WHAT COMES NEXT

Lesson 5.2: **CNNs** — the conv layers from our convolutional autoencoder become the backbone of an entirely new architecture for spatial data.

LESSON 5.1: REAL-WORLD APPLICATIONS

An autoencoder that cannot reconstruct an input is telling you that input does not look like what it was trained on. That is the single idea behind every application on this slide.

Finance — Credit Card Fraud (DBS, UOB, OCBC)

Train a vanilla or denoising AE on millions of *normal* card transactions. At runtime, a transaction whose reconstruction error exceeds a threshold is flagged for review. Works without any labelled fraud examples — critical, because less than 0.1% of real transactions are ever labelled as fraud.

Manufacturing — Defect Detection on the Line

A Singapore semiconductor fab trains a conv-AE on wafer images of good dies. When the reconstruction error exceeds a threshold, the wafer is routed for manual inspection. No one has to hand-label every defect class — the AE learns what "normal" looks like and flags the rest.

Healthcare — Medical Imaging Anomalies (SGH, NUH)

Train a convolutional AE on thousands of *normal* chest X-rays. Lesions, tumours, or effusions produce high reconstruction error because the model has never seen their texture. A radiologist triages from a ranked anomaly list instead of scanning every image.

Telecom — Network Intrusion Detection (Singtel, StarHub)

Train an AE on normal network-flow statistics. Attacker traffic — scans, exfiltration bursts, lateral movement — reconstructs poorly and triggers an alert. The same principle protects VAEs used for synthetic healthcare data, where sampling from the latent prior generates privacy-preserving patient records.

Why this matters for YOU: when the business says "we have only the *good* examples, not the bad ones," an autoencoder is the right first tool — trade a supervised classifier (needs labelled fraud) for a reconstruction threshold (needs only normal data).

LESSON 5.2

CNNS AND COMPUTER VISION

Spatial Feature Learning — Extracting Patterns from Grid Data

"What if the model could see edges, textures, and objects — the way you do?"

THE CONVOLUTION OPERATION

HOW IT WORKS

Filter (kernel): small matrix (e.g., 3x3) sliding over input

Stride: how far the filter moves each step

Padding: zeros added around border to control output size

Feature map: the output after applying one filter

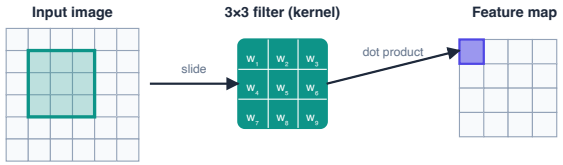
Analogy: A magnifying glass scanning across a photo, detecting one specific pattern at every position.

OUTPUT SIZE FORMULA

$$O = \frac{W - F + 2P}{S} + 1$$

W = input, F = filter, P = padding, S = stride

Example: Input 28x28, filter 3x3, padding 1, stride 1: $(28 - 3 + 2)/1 + 1 = 28$. Same spatial dimensions — "same" padding.



$y[i,j] = \Sigma_{km}\, w[m,n] \cdot x[i+m, j+n]$

Output size: (W – F + 2P) / S + 1

Same filter weights everywhere → translation invariance

POOLING AND FEATURE HIERARCHY

POOLING LAYERS

Max pooling: take the maximum value in each window — preserves strongest activation

Average pooling: take the mean — preserves overall signal

Global average pooling (GAP): average entire feature map to a single value — replaces fully connected layers

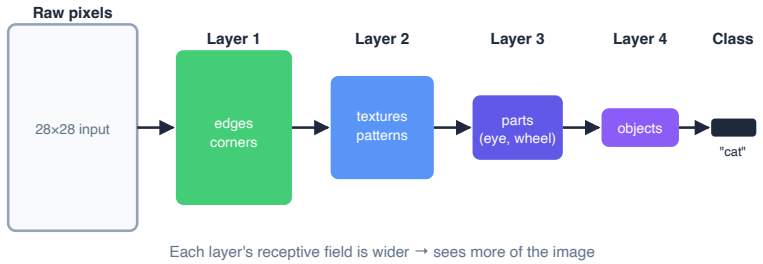
HIERARCHICAL FEATURE LEARNING

Early layers: edges, textures, colours

Middle layers: parts, patterns, shapes

Late layers: objects, faces, scenes

> **Why this matters:** Each layer builds on the previous. Low-level features compose into high-level concepts. This is why CNNs transfer well — early features are universal.



Each layer's receptive field is wider → sees more of the image

CNN ARCHITECTURE EVOLUTION

Architecture	Year	Key Innovation	Depth
LeNet-5	1998	First practical CNN (handwritten digits)	5
AlexNet	2012	ReLU, dropout, GPU training	8
VGGNet	2014	Very small 3x3 filters, depth over width	16-19
GoogLeNet	2014	Inception module — parallel filter sizes	22
ResNet	2015	Skip connections — trains 152+ layers	50-152

The depth problem: VGG showed deeper = better, but training became unstable beyond ~20 layers. ResNet solved this with skip connections, enabling 152-layer networks that outperform shallower ones.

RESNET: SKIP CONNECTIONS

$$\mathcal{H}(x) = \mathcal{F}(x) + x$$

Output = learned residual + identity shortcut

WHY IT WORKS

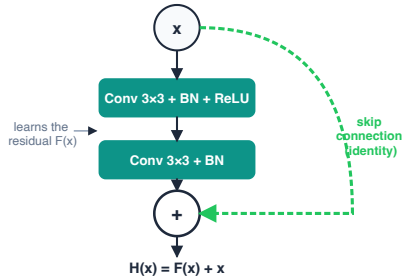
The network learns the *residual* $\mathcal{F}(x) = \mathcal{H}(x) - x$ rather than the full mapping. If the optimal mapping is close to identity, learning a small residual is easier than learning the full function.

GRADIENT FLOW

$\frac{\partial \mathcal{H}}{\partial x} = \frac{\partial \mathcal{F}}{\partial x} + 1$. The **+1** ensures gradients never vanish completely — they always have the identity path.

```python
class ResBlock(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.conv1 = nn.Conv2d(channels,
            channels, 3, padding=1)
        self.bn1 = nn.BatchNorm2d(channels)
        self.conv2 = nn.Conv2d(channels,
            channels, 3, padding=1)
        self.bn2 = nn.BatchNorm2d(channels)

    def forward(self, x):
        residual = x
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        out += residual  # skip connection
        return F.relu(out)
```



MODERN ENHANCEMENT: SE BLOCKS

SQUEEZE-AND-EXCITATION

$$s = \sigma(W_2 \cdot \mathrm{ReLU}(W_1 \cdot \mathrm{GAP}(x)))$$

Channel attention weights via global average pooling

SQUEEZE

Global average pooling compresses each channel to a single value

EXCITATION

Two FC layers learn which channels are important for this input

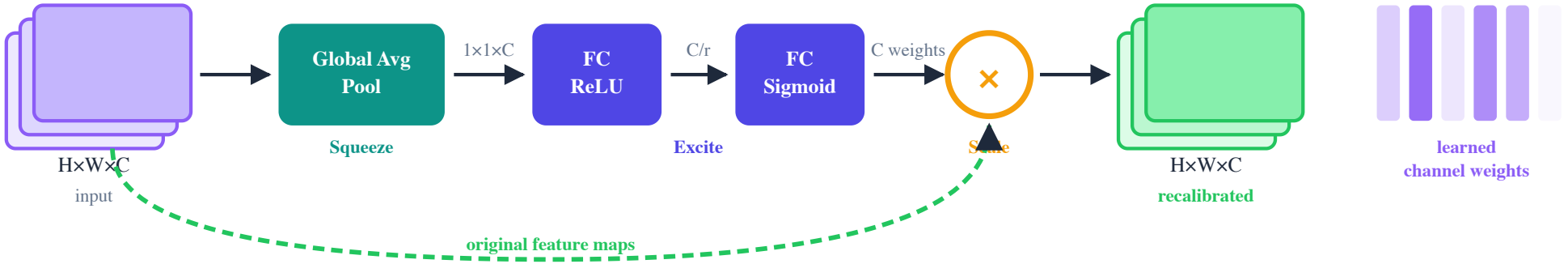
RECALIBRATE

Multiply feature maps by learned channel weights

```python
class SEBlock(nn.Module):
    def __init__(self, channels, reduction=16):
        super().__init__()
        self.squeeze = nn.AdaptiveAvgPool2d(1)
        self.excite = nn.Sequential(
            nn.Linear(channels,
                      channels // reduction),
            nn.ReLU(),
            nn.Linear(channels // reduction,
                      channels),
            nn.Sigmoid())

    def forward(self, x):
        b, c, _, _ = x.size()
        s = self.squeeze(x).view(b, c)
        s = self.excite(s).view(b, c, 1, 1)
        return x * s  # recalibrate
```

SE BLOCK: DATA FLOW



Reading the diagram: The input tensor (purple, H×W×C) is squeezed to a 1×1×C vector by global average pooling. Two FC layers learn per-channel importance weights (the purple bars on the right — taller = more important). The original feature maps are multiplied by these weights, boosting relevant channels and suppressing irrelevant ones.

MODERN TRAINING ENHANCEMENTS

INITIALISATION

Kaiming initialisation: $W \sim \mathcal{N}(0, \quad 2/n_{\text{in}})$

Designed for ReLU networks. Prevents signal from vanishing or exploding through layers.

MIXED PRECISION TRAINING

Forward pass in FP16 (fast, less memory)

Loss scaling to prevent FP16 underflow

Weight updates in FP32 (full precision)

~2x speedup on modern GPUs

DATA AUGMENTATION

Mixup: $\tilde{x} = \lambda x_i + (1 - \lambda) x_j$

Blend two training examples and their labels. Smoother decision boundaries, better calibration.

LABEL SMOOTHING

Target: $y_{\text{smooth}} = (1 - \alpha) \cdot y + \alpha / K$

Prevents overconfident predictions. $\alpha = 0.1$ typical. K = number of classes.

ADVANCED

VISION TRANSFORMERS (VIT) — BRIEF INTRODUCTION

CNN VS VIT

Property	CNN	ViT
Inductive bias	Strong (locality)	Weak
Small data	Better	Worse
Large data	Good	Better
Global context	Late layers only	Every layer

Key result: ViT outperforms CNNs on large datasets (ImageNet-21k). CNNs win on small datasets due to inductive biases (translation invariance, locality).

Full transformer derivation in Lesson 5.4.

KAILASH BRIDGE: ONNXBRIDGE

FROM TRAINING TO DEPLOYMENT

```python
from kailash_ml import OnnxBridge

bridge = OnnxBridge()

# Export trained model to ONNX
bridge.export(
    model=resnet,
    input_shape=(1, 1, 28, 28),
    output_path="resnet_fmnist.onnx"
)

# Verify exported model
result = bridge.validate(
    "resnet_fmnist.onnx"
)
print(result.status)  # "valid"
```

WHY ONNX?

Framework-agnostic: train in PyTorch, deploy anywhere

Optimised inference: ONNX Runtime is faster than native PyTorch

Edge deployment: run on mobile, browser, embedded

> **Pattern:** Train with PyTorch → Export with OnnxBridge → Serve with InferenceServer (Lesson 5.7).

EXERCISE 5.2: CNN CLASSIFICATION PIPELINE

TASKS

- Build a simple CNN for Fashion-MNIST classification
- Add ResBlock with skip connections — compare accuracy
- Add SE block — measure improvement
- Apply Mixup augmentation and label smoothing
- Export final model to ONNX with OnnxBridge

ASSESSMENT CRITERIA

- Architecture progressively improved (baseline → ResBlock → SE)
- Training enhancements measurably improve performance
- Training curves visualised with ModelVisualizer
- ONNX export successful and validated

LESSON 5.2 KEY FORMULAS

$$O = \frac{W - F + 2P}{S} + 1$$

Conv output size

$$\mathcal{H}(x) = \mathcal{F}(x) + x$$

ResNet skip connection

$$s = \sigma(W_2 \cdot \mathrm{ReLU}(W_1 \cdot \mathrm{GAP}(x)))$$

SE block channel attention

$$\tilde{x} = \lambda x_i + (1 - \lambda) x_j$$

Mixup augmentation

LESSON 5.2 SUMMARY

CNNs extract spatial features hierarchically: edges → textures → objects. Skip connections enable depth. Modern enhancements push accuracy further.

KEY TAKEAWAYS

Conv output size: $(W - F + 2P)/S + 1$

ResNet enables 150+ layer networks

SE blocks add channel attention cheaply

OnnxBridge exports models for deployment

WHAT COMES NEXT

Lesson 5.3: **RNNs and Sequence Models** — from spatial features (grids) to temporal features (sequences). The hidden state replaces the feature map.

LESSON 5.2: REAL-WORLD APPLICATIONS

The same CIFAR-10 pipeline you just built — convolutions, residual blocks, SE attention, ONNX export — powers production image classifiers across every industry that has cameras.

Healthcare — Chest X-Ray Triage

ResNet with SE blocks, fine-tuned on a Singapore public-health chest X-ray corpus, classifies scans into "normal / pneumonia / tuberculosis / other" and ranks the worst cases to the top of the radiologist's queue. The same architecture you wrote in Exercise 2, trained on medical images instead of CIFAR.

Manufacturing — Automated Quality Control

A Singapore precision-engineering firm uses a ResNet variant to grade machined parts on a conveyor belt. Mixed-precision training halved GPU cost during weekly retraining; SE blocks improved recall on a rare defect class from 71% to 89%. The trained model is exported to ONNX and served on the factory-floor edge server.

Retail — Shelf Monitoring at FairPrice, Cold Storage

Cameras above each aisle feed a CNN that counts SKUs, detects out-of-stocks, and identifies misplaced items. The store manager gets a dashboard ranked by lost-sales-per-hour, not a 200-aisle walkthrough. Latency budget: under 200ms per frame — exactly the kind of workload ONNX export was built for.

Public Safety — MRT Crowd Density from CCTV

SMRT platforms use CNN-based crowd counting from existing CCTV feeds to predict overcrowding before it happens, triggering train dispatch or gate throttling. The output of the last conv layer is a density map, not a single number — the same "later layers = higher-level features" hierarchy you learned.

Why this matters for YOU: every production CNN starts with three decisions from Exercise 2 — how deep, which residual pattern, and whether attention earns its compute. A 2% gain from SE blocks is often the difference between a model that ships and one that does not.

LESSON 5.3

RNNS AND SEQUENCE MODELS

Temporal Feature Learning — Extracting Patterns from Sequences

"What if the model could remember what it saw five steps ago?"

RECURRENT NEURAL NETWORKS

CORE CONCEPT

Hidden state: memory that persists across time steps

At each step, combine current input with previous hidden state

Same weights applied at every time step (weight sharing)

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b)$$

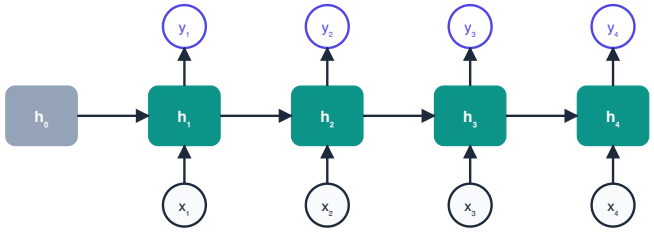
Hidden state update at time t

THE PROBLEM: VANISHING GRADIENTS

Gradients shrink exponentially as they backpropagate through time steps. After ~10-20 steps, gradients are effectively zero. The network "forgets" early inputs.

Formally: $\frac{\partial h_T}{\partial h_1} = \prod_{t=1}^{T-1} \frac{\partial h_{t+1}}{\partial h_t}$. If each factor <1, the product vanishes.

Solution: LSTM and GRU (next slides).



Same weights W reused at every time step → $h_t = \tanh(W_h \cdot x_t + W_h \cdot h_{t-1})$

LSTM: LONG SHORT-TERM MEMORY

FOUR COMPONENTS

Cell state C_t — the "highway" that carries information across time steps with minimal modification

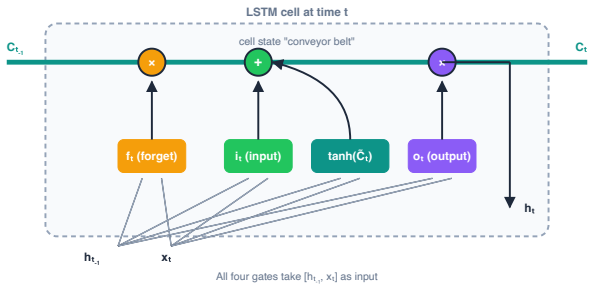
Forget gate f_t — decides what to discard from cell state

Input gate i_t — decides what new information to store

Output gate o_t — decides what to output from cell state

Analogy: The cell state is a conveyor belt running through the factory. Gates are workers who add items (input gate), remove items (forget gate), and select items for the next station (output gate). The belt itself flows freely.

Why it solves vanishing gradients: The cell state path has *additive* interactions (not multiplicative). Gradients flow through addition, not repeated multiplication, preventing exponential decay.



LSTM: ALL FIVE EQUATIONS

FORGET GATE

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Sigmoid output in [0,1]. Close to 0 = forget. Close to 1 = remember.

INPUT GATE

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

Controls how much of the new candidate to add.

CELL UPDATE

$$C_t = f_t \odot C_{t-1} + i_t \odot \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Forget old + add new. The additive structure preserves gradients.

LSTM: OUTPUT GATE AND HIDDEN STATE

OUTPUT GATE

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

Controls which parts of the cell state become the output.

HIDDEN STATE

$$h_t = o_t \odot \tanh(C_t)$$

The hidden state is a filtered, squashed version of the cell state. This is the output at time t.

5 equations, 4 gates: forget (f_t), input (i_t), cell update (C_t), output (o_t), hidden state (h_t). Every gate takes the same inputs: $[h_{t-1}, x_t]$. The difference is the learned weight matrices.

Parameter count: For input size d and hidden size h, an LSTM has $4(d \cdot h + h \cdot h + h)$ parameters — four weight matrices, four bias vectors.

GRU: GATED RECURRENT UNIT

SIMPLIFIED LSTM

Merges cell state and hidden state into one

Two gates instead of three: **update** and **reset**

Fewer parameters, faster training

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

Update gate (combines forget + input)

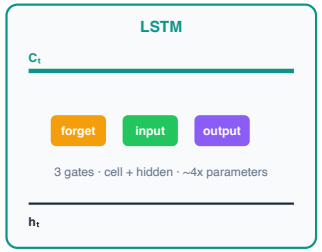
$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

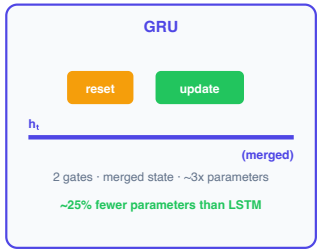
Reset gate (controls previous state influence)

LSTM VS GRU

Property	LSTM	GRU
Gates	3 + cell	2
Parameters	More	~25% fewer
Long sequences	Better	Comparable
Small data	Overfits more	Better
Speed	Slower	Faster

Rule of thumb: Start with GRU. Switch to LSTM if performance is insufficient on long sequences.





ATTENTION MECHANISMS FOR SEQUENCES

TEMPORAL ATTENTION

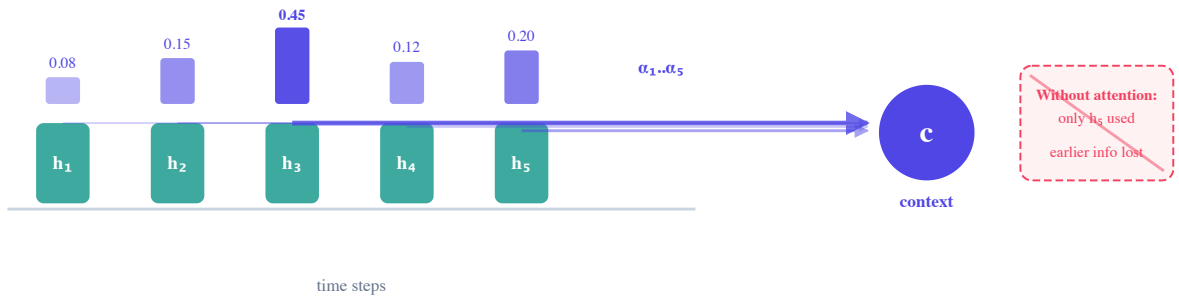
Instead of using only the final hidden state, **attend to all time steps** with learned importance weights.

$$\alpha_t = \frac{\exp(e_t)}{\sum_{k=1}^{T} \exp(e_k)}$$

Attention weight for time step t (softmax over scores)

$$c = \sum_{t=1}^{T} \alpha_t h_t$$

Context vector = weighted sum of hidden states



WHY ATTENTION?

Without attention: the model must compress an entire sequence into a single fixed-size vector (the last hidden state). Long sequences lose early information.

With attention: the model can "look back" at any time step. The attention weights are interpretable — you can see which inputs the model focuses on.

This is the precursor to self-attention in Transformers (Lesson 5.4).

SEQUENCE MODEL METRICS

PERPLEXITY

$$\text{PP} = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log P(w_i)\right)$$

Lower is better. PP=1 means perfect prediction.

Measures how "surprised" the model is by the test data. A perplexity of 50 means the model is as uncertain as choosing uniformly among 50 options.

OTHER METRICS

Metric	Task	Measures
BLEU	Translation	n-gram overlap
Cross-entropy	Classification	Log probability of correct class
Sequence accuracy	Seq-to-seq	Exact match rate
MAE/RMSE	Time series	Prediction error

APPLICATIONS: FINANCE AND TEXT

FINANCIAL TIME SERIES

Technical indicators as features: RSI, MACD, Bollinger Bands

LSTM captures temporal patterns in price movements

Multi-step forecasting with teacher forcing

Singapore stock market data (SGX)

TEXT GENERATION

Character-level LSTM: predict next character

Train on Shakespeare → generate Shakespeare-like text

Temperature parameter controls randomness

Sampling strategy: greedy, top-k, nucleus

Preview: This is exactly how GPT works, but with transformers instead of LSTMs and at much larger scale.

EXERCISE 5.3: SEQUENCE MODELLING

TASKS

- Build LSTM for Singapore stock price prediction with technical indicators
- Add temporal attention layer — visualise attention weights
- Compare LSTM vs GRU (accuracy and training speed)
- Implement character-level text generation with LSTM

ASSESSMENT CRITERIA

- LSTM gate equations implemented correctly
- Attention improves prediction (quantified)
- GRU vs LSTM comparison documented
- Text generation produces coherent output

Gradient clipping: Essential for RNN training. Use `torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)`

LESSON 5.3 SUMMARY

LSTMs solve the vanishing gradient problem with gated cell states. Attention lets the model focus on relevant time steps. GRUs offer a simpler alternative.

KEY EQUATIONS (5)

Forget: $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$

Input: $i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$

Cell: $C_t = f_t \odot C_{t-1} + i_t \odot \tanh(W_C[h_{t-1}, x_t] + b_C)$

Output: $o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$

Hidden: $h_t = o_t \odot \tanh(C_t)$

WHAT COMES NEXT

Lesson 5.4: **Transformers** — what if we replaced the sequential hidden state with *pure attention*? No recurrence. No sequential bottleneck. Process the entire sequence in parallel.

LESSON 5.3: REAL-WORLD APPLICATIONS

The multi-stock forecaster you built in Exercise 3 — LSTM with temporal attention, trained on real STI + APAC tickers — is the same architecture used for every business problem where "what happens next" depends on "what has been happening".

Finance — SGX Price Forecasting & Fraud Sequences

Quant desks at DBS and UOB run LSTM+attention models on minute-bar SGX data with the exact feature set from Exercise 3 (RSI, MACD, Bollinger Bands). The same architecture, trained on transaction sequences instead of prices, detects card-not-present fraud rings by their spending *cadence* — something a point-in-time classifier cannot see.

Healthcare — ICU Vital Sign Early Warning

SGH's ICU deploys LSTMs over streams of heart rate, blood pressure, SpO2, and respiratory rate to predict deterioration 6–12 hours before it is clinically obvious. Temporal attention highlights *which* time steps mattered, giving clinicians an interpretable "why" alongside the alert.

Retail — FMCG Demand Forecasting

A Singapore distributor forecasts weekly SKU-level demand across 3,000 products using GRU networks (fewer parameters, faster training than LSTM — exactly the trade-off you measured in Exercise 3). Promo calendars and public holidays enter as auxiliary sequence features.

Transport — MRT Ridership Prediction

LTA forecasts MRT ridership per station per 15-minute bin using sequence models; operations uses the forecast to pre-position trains. Attention weights show the model leaning on "the same 07:45 slot from last week" during normal days and on "the last 30 minutes" when rain is falling — interpretable behaviour switching.

Why this matters for YOU: when the question is "given what has happened, what happens next?", sequence models are your starting point. The business value is the *lead time* — six hours on ICU deterioration, a week on stock-outs, 30 min on crowding — converting emergencies into scheduled decisions.

LESSON 5.4

TRANSFORMERS

Attention-Based Feature Learning — Processing Sequences in Parallel

"Attention is all you need." — Vaswani et al., 2017

SELF-ATTENTION: THE CORE IDEA

THREE PROJECTIONS

Query (Q): "What am I looking for?"
Key (K): "What do I contain?"
Value (V): "What information do I carry?"

Each token projects its embedding into Q, K, and V via learned weight matrices.

Analogy: A library search. Your Query is your question. Each book has a Key (title/index). You compare your Query with all Keys to find the best matches. Then you read the Values (content) of the matching books.

Key difference from RNN attention: In RNN attention, queries come from the decoder and keys/values from the encoder. In *self*-attention, Q, K, V all come from the same sequence — each token attends to every other token.

THEORY

SCALED DOT-PRODUCT ATTENTION

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{d_k}\right)V$$

The fundamental equation of modern AI

STEP 1: COMPUTE SCORES

$QK^\top$ — dot product between every query and every key. High dot product = high similarity.

STEP 2: SCALE

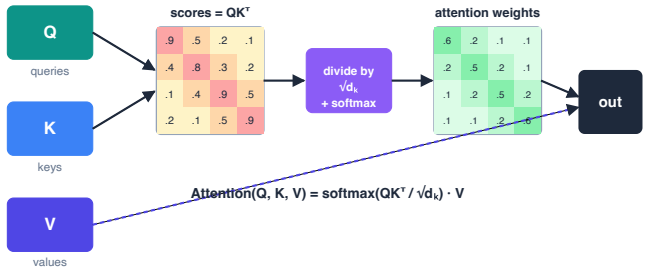
Divide by d_k. Without scaling, large d_k produces large dot products, pushing softmax into saturation (all mass on one token).

STEP 3: NORMALISE

Softmax converts scores to probabilities that sum to 1 across keys. Each query gets a probability distribution over all tokens.

STEP 4: WEIGHTED SUM

Multiply attention weights by Values. Each token's output is a weighted combination of all tokens' values.



WHY DIVIDE BY D_K?

THE PROBLEM

If $q_i, k_j \sim \mathcal{N}(0,1)$, then $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ has variance d_k.

WHY IT MATTERS

Large variance → extreme values → softmax saturates → gradients vanish. For $d_k = 512$, dot products have std dev $\sqrt{512} \approx 22.6$.

THE FIX

Dividing by $\sqrt{d_k}$ normalises variance back to 1. Softmax operates in a healthy range. Gradients flow.

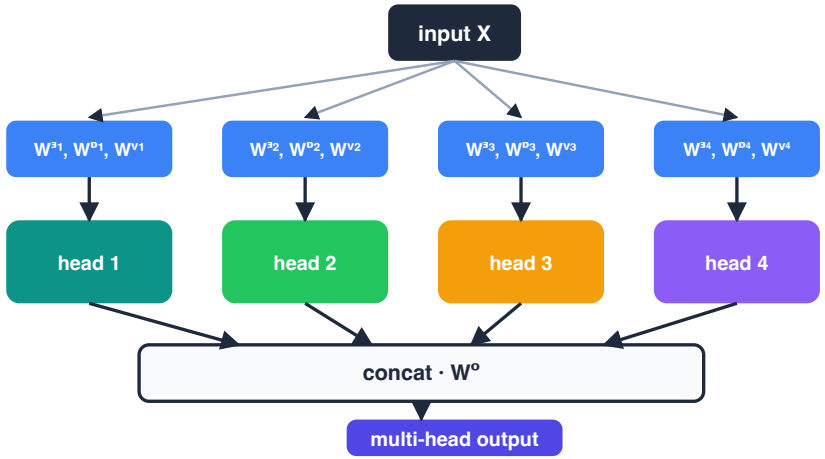
Proof sketch:
If q_i, k_j are i.i.d. with mean 0 and variance 1:
$\text{Var}(q \cdot k) = \sum_{i=1}^{d_k} \text{Var}(q_i k_i) = d_k$
Dividing by $\sqrt{d_k}$:
$\text{Var}\left(\frac{q \cdot k}{\sqrt{d_k}}\right) = \frac{d_k}{d_k} = 1$

Interview question: "What happens if you use d_k instead of $\sqrt{d_k}$?" Answer: variance becomes $1/d_k$ — too small, softmax becomes too uniform, attention is diluted.

MULTI-HEAD ATTENTION

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$



WHY MULTIPLE HEADS?

Each head learns a **different relationship**

Head 1 → syntactic structure

Head 2 → semantic similarity

Head 3 → positional proximity

DIMENSIONS

$d_{\text{model}} = 512$, heads $h = 8$

Per-head $d_k = d_v = d_{\text{model}}/h = 64$

Same compute as single-head with $d_k = 512$

Key insight: multi-head is NOT more expensive — dimensions split across heads, W^O recombines.

POSITIONAL ENCODING

THE PROBLEM

Self-attention is **permutation-invariant**. "The cat sat on the mat" and "mat the on sat cat the" produce the same attention scores. Position information is lost.

THE SOLUTION: SINUSOIDAL ENCODING

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

WHY SINUSOIDAL?

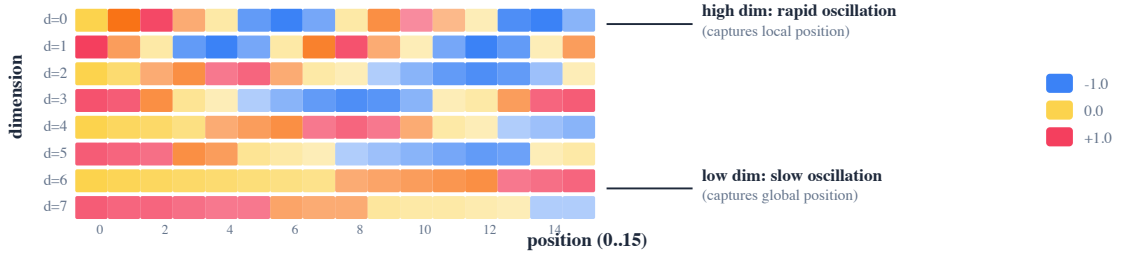
Unique: each position gets a distinct encoding

Relative positions: for any fixed offset k, PE_{pos+k} is a linear function of PE_{pos}

Bounded: values stay in [-1, 1]

Extrapolation: works for sequences longer than training

Alternative: Learned positional embeddings (BERT, GPT). RoPE (Rotary Position Embedding) in modern LLMs. ALiBi for length extrapolation.



TRANSFORMER ARCHITECTURE

ENCODER BLOCK

Multi-head self-attention

Add & Layer Norm (residual connection)

Feed-forward network (2 layers, ReLU)

Add & Layer Norm (residual connection)

Stack N encoder blocks (N=6 in original paper).

DECODER BLOCK

Masked multi-head self-attention (cannot attend to future tokens)

Add & Layer Norm

Cross-attention to encoder output (Q from decoder, K/V from encoder)

Add & Layer Norm

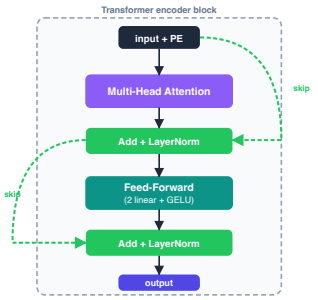
Feed-forward network

Add & Layer Norm

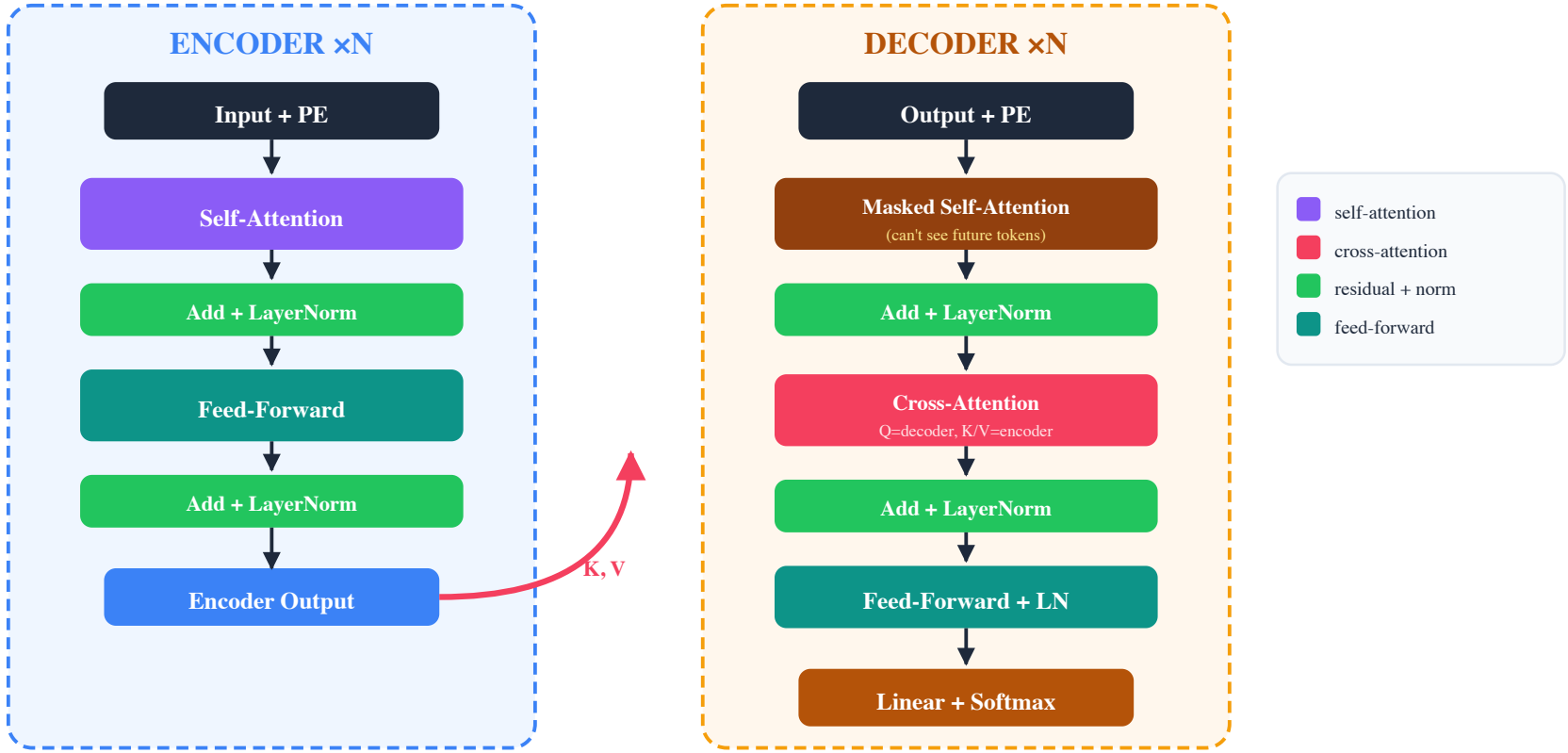
Stack N decoder blocks (N=6 in original paper).

WHY LAYER NORM (NOT BATCH NORM)?

Batch norm normalises across the batch. But sequence lengths vary — batch statistics are meaningless. Layer norm normalises across features for each token independently.



ENCODER-DECODER: THE FULL PICTURE



The **cross-attention** layer is the bridge: the decoder's queries ask "what in the encoder output is relevant to what I'm generating next?"

ADVANCED

TRANSFORMER VARIANTS

Model	Architecture	Training Objective	Best For
BERT	Encoder only	Masked language modelling + NSP	Understanding (NLU, classification)
GPT	Decoder only	Autoregressive next-token	Generation (text, code)
T5	Encoder-decoder	Text-to-text (unified)	Any NLP task
Transformer-XL	Decoder + recurrence	Segment-level memory	Long contexts
Reformer	Efficient attention	LSH attention + reversible layers	Very long sequences

The landscape in 2024+: Decoder-only (GPT-style) dominates. BERT-style encoders still preferred for classification/retrieval. Efficient attention (Flash Attention, Ring Attention) makes long contexts practical without architectural changes.

CONSOLIDATION: FOUR PARADIGMS COMPARED

After 4 lessons, you have seen all major DL paradigms. Which architecture for which data?

Data Type	Best Architecture	Why	Lesson
Images	CNN / ViT	Spatial hierarchy / global attention	5.2
Sequences	RNN / Transformer	Temporal memory / parallel attention	5.3, 5.4
Graphs	GNN	Neighbourhood aggregation	5.6
Generation	VAE / GAN / Diffusion	Latent sampling / adversarial / denoising	5.1, 5.5
Text (NLU)	BERT (Encoder)	Bidirectional context	5.4
Text (Gen)	GPT (Decoder)	Autoregressive generation	5.4

EXERCISE 5.4: TRANSFORMERS

TASKS

- Derive scaled dot-product attention from scratch (pen and paper + code)
- Implement single-head attention in PyTorch
- Fine-tune BERT for text classification (TREC-6 dataset)
- Compare BERT vs LSTM baseline from 5.3 (accuracy, training time)

ASSESSMENT CRITERIA

- Self-attention implemented correctly from scratch
- BERT fine-tuning produces strong classification accuracy
- BERT vs LSTM comparison quantified
- Can explain why d_k scaling is necessary

THEORY

LESSON 5.4 KEY FORMULAS

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{d_k}\right)V$$

Scaled dot-product attention

$$\text{MultiHead} = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O$$

Multi-head attention

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

Positional encoding (even dimensions)

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

Positional encoding (odd dimensions)

LESSON 5.4 SUMMARY

Transformers process sequences in parallel via self-attention. No recurrence, no convolution — just attention. The foundation of modern AI.

KEY TAKEAWAYS

Self-attention: every token attends to every other token

Scale by d_k to prevent softmax saturation

Multi-head = diverse attention at no extra cost

Positional encoding provides order information

WHAT COMES NEXT

Lesson 5.5: **GANs and Diffusion** — from understanding data to *generating* data. The VAE from 5.1 was our first generative model. Now we add adversarial training.

LESSON 5.4: REAL-WORLD APPLICATIONS

The BERT fine-tuning pipeline you ran on AG News in Exercise 4 is the same pipeline used everywhere that text has to be classified, summarised, or routed at scale. Swap the dataset, keep the architecture.

Finance — Earnings Call & SEC Filing Sentiment

Sell-side analysts fine-tune BERT on labelled earnings-call transcripts to score each quarter as beat / miss / guidance-cut — in the first minute after the call ends, not after a human has read it. A MAS-supervised fund house fine-tunes the same model on SGX filings to flag risk language (e.g., "going concern", "material weakness") for the credit desk.

Customer Service — IRAS Tax Query Routing

A public-sector contact centre fine-tunes BERT on historical tickets to classify incoming queries into 40+ tax-topic buckets and route them to the right specialist. Accuracy on the first 20 classes exceeds the human baseline; the remaining 20 are human-in-the-loop. That is the *exact* 4-class AG News fine-tune you just ran, scaled up and domain-swapped.

Healthcare — Clinical Note Summarisation

Fine-tuned T5 and BART models at public hospitals convert 2,000-word admission notes into 200-word discharge summaries. The clinician edits the summary instead of writing it from scratch — saving 15 minutes per patient across 2M admissions a year. The self-attention mechanism is the same one you derived by hand.

Legal — Contract Review Assistance

Singapore law firms use fine-tuned transformers to extract obligations, liabilities, and termination clauses from draft contracts. The model highlights the spans; the associate reviews a ranked list. Multi-head attention visualisations are the interpretability layer — the partner wants to see *why* the model flagged a clause.

Why this matters for YOU: fine-tuning a pre-trained transformer is the cheapest high-impact ML project most organisations can run. Data requirements drop 10–100×, training drops from weeks to hours, and the model is production-ready — the biggest ROI in M5.

LESSON 5.5

GENERATIVE MODELS — GANS AND DIFFUSION

Generative Modelling — Learning to Create New Data

"What if the model could imagine things it has never seen?"

GAN: GENERATOR VS DISCRIMINATOR

THE GAME

Generator (G): takes random noise, produces fake data

Discriminator (D): takes data (real or fake), outputs probability of "real"

G wants to fool D. D wants to catch G.

Both improve through competition

Analogy: A forger (G) creates fake paintings. An art critic (D) tries to detect forgeries. Over time, the forger becomes so good that the critic cannot tell the difference.

TRAINING LOOP

Sample real data batch from dataset

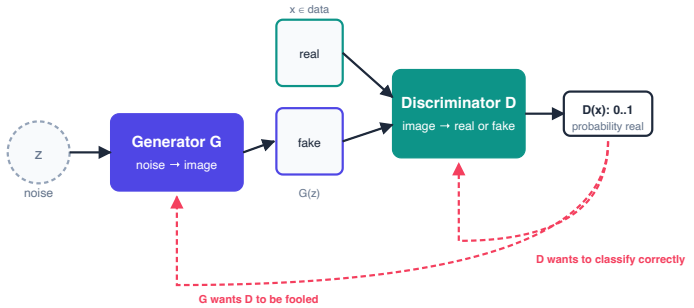
Sample noise, generate fake data with G

Train D to distinguish real from fake

Train G to fool D (maximise D's error on fakes)

Repeat — alternating optimisation

Equilibrium: Ideally, G produces perfect fakes and D outputs 0.5 for everything (cannot tell). In practice, this equilibrium is fragile.



GAN MINIMAX OBJECTIVE

$$\min_G \max_D \; \mathbb{E}_{x \sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z \sim p_z}[\log(1 - D(G(z)))]$$

Generator minimises, Discriminator maximises — zero-sum game

DISCRIMINATOR'S VIEW

Maximise: $\log D(x)$ should be high (correctly classify real) and $\log(1 - D(G(z)))$ should be high (correctly reject fake).

GENERATOR'S VIEW

Minimise: wants $D(G(z)) \to 1$, making $\log(1 - D(G(z))) \to -\infty$. In practice, maximise $\log D(G(z))$ instead (better gradients).

Theoretical optimum: When $p_G = p_{\text{data}}$, the optimal discriminator is $D^*(x) = 1/2$ everywhere, and the minimax value is $-\log 4$.

DCGAN: DEEP CONVOLUTIONAL GAN

ARCHITECTURE GUIDELINES

- Replace all pooling with strided convolutions
- Use batch normalisation in both G and D
- Remove fully connected layers (use conv throughout)
- Generator: ReLU activation, Tanh output
- Discriminator: LeakyReLU activation

```
class Generator(nn.Module):
    def __init__(self, nz=100, ngf=64):
        super().__init__()
        self.main = nn.Sequential(
            nn.ConvTranspose2d(nz, ngf*4,
                4, 1, 0, bias=False),
            nn.BatchNorm2d(ngf*4),
            nn.ReLU(True),
            nn.ConvTranspose2d(ngf*4, ngf*2,
                4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf*2),
            nn.ReLU(True),
            # ... more layers
            nn.Tanh())
```

WGAN: WASSERSTEIN GAN

$$\min_G \max_{D \in \mathcal{D}} \mathbb{E}_{x \sim p_{\text{data}}}[D(x)] - \mathbb{E}_{z \sim p_z}[D(G(z))]$$

Wasserstein distance — no log, no sigmoid, critic not classifier

KEY DIFFERENCES FROM STANDARD GAN

Critic, not discriminator: outputs a real-valued score, not a probability

Wasserstein distance instead of JS divergence — provides meaningful gradients even when distributions don't overlap

Gradient penalty enforces Lipschitz constraint (WGAN-GP)

WHY WGAN IS BETTER

Mode collapse prevention: Standard GANs can collapse to generating one image. Wasserstein distance provides gradients that push toward covering the full data distribution.

Training stability: The Wasserstein loss correlates with generation quality. You can actually track convergence — unlike standard GAN loss which oscillates meaninglessly.

ADVANCED

GAN VARIANTS (SURVEY)

Variant	Key Innovation	Application
Conditional GAN	Condition on class labels for controlled generation	Generate specific digit/class
CycleGAN	Unpaired image-to-image translation (cycle consistency)	Horse to zebra, photo to painting
StyleGAN	Style-based progressive generation, AdaIN layers	High-resolution face generation
Pix2Pix	Paired image-to-image translation	Sketch to photo, map to satellite

Evaluation metrics: FID (Frechet Inception Distance) measures distribution similarity. IS (Inception Score) measures quality and diversity. Lower FID is better. Higher IS is better.

$$\text{FID} = \|\mu_r - \mu_g\|^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2})$$

Frechet Inception Distance — compares real (r) and generated (g) feature distributions

ADVANCED

DIFFUSION MODELS (BRIEF)

CORE IDEA: DDPM

Forward process: gradually add Gaussian noise over T steps until data becomes pure noise

Reverse process: learn to denoise step by step, recovering the original data

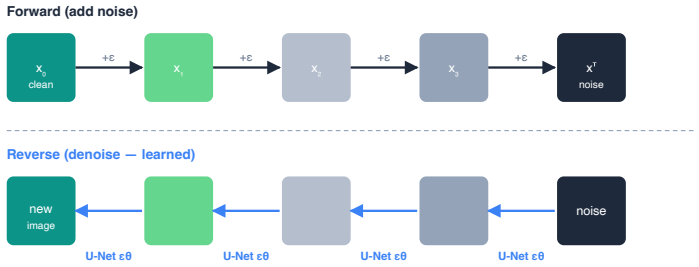
Train a neural network to predict the noise at each step

Advantage over GANs: More stable training (no adversarial dynamics). Better mode coverage (no mode collapse). Higher quality at the cost of slower generation.

WHEN TO USE WHAT

Method	Best For
VAE	Fast generation, smooth latent space
GAN	Sharp images, real-time generation
Diffusion	Highest quality, diverse outputs

Stable Diffusion: Uses a VAE for latent space + diffusion in latent space + text conditioning via CLIP. The current state of the art for text-to-image generation.



GAN TRAINING CHALLENGES

MODE COLLAPSE

Symptom: Generator produces the same output regardless of input noise. It found one image that fools the discriminator and keeps producing it.

SOLUTIONS

WGAN: Wasserstein distance encourages full coverage

Minibatch discrimination: D sees batch statistics

Spectral normalisation: stabilise D's Lipschitz constant

TRAINING INSTABILITY

Symptom: Loss oscillates wildly. G and D alternate between dominance. No convergence.

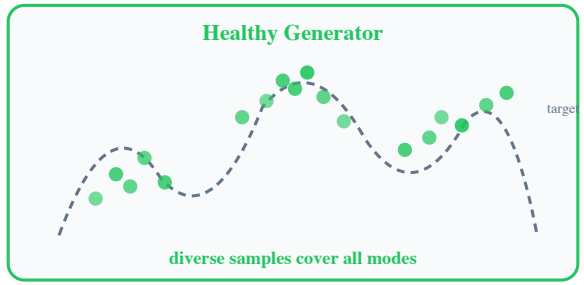
BEST PRACTICES

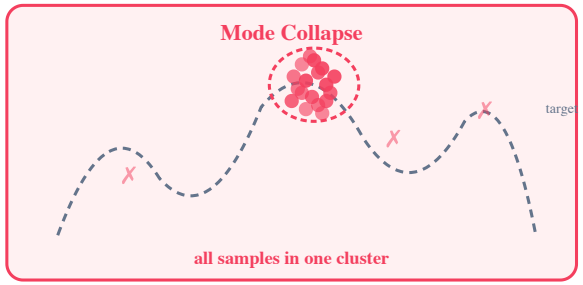
Use Adam with low learning rate (0.0002)

Train D more steps than G (e.g., 5:1 ratio)

Use label smoothing (real labels = 0.9, not 1.0)

Monitor FID during training, not just loss





EXERCISE 5.5: GENERATIVE MODELS

TASKS

- Implement DCGAN for MNIST image generation
- Implement WGAN-GP — compare training stability
- Compute FID score for both models
- Document: when to use GANs vs VAE vs diffusion

ASSESSMENT CRITERIA

- DCGAN generates recognisable digits
- WGAN is demonstrably more stable (loss curves)
- FID computed and compared
- Generation model selection guide understood

LESSON 5.5 SUMMARY

GANs learn to generate data through adversarial competition. WGAN stabilises training. Diffusion models offer higher quality at higher cost.

KEY EQUATIONS

GAN: $\min_G \max_D [\mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]]$

WGAN: $\min_G \max_D [\mathbb{E}[D(x)] - \mathbb{E}[D(G(z))]]$

WHAT COMES NEXT

Lesson 5.6: **Graph Neural Networks** — what if your data is not a grid (images) or a sequence (text) but a graph (social networks, molecules)?

LESSON 5.5: REAL-WORLD APPLICATIONS

DCGAN, WGAN-GP, and diffusion — the three generators you built in Exercise 5 — solve a class of problems where the business constraint is "we need more data, but we cannot collect it". Synthetic generation converts a hard collection problem into an ML problem.

Healthcare — Privacy-Preserving Training Data

SGH cannot share patient CT scans with a research lab overseas, but it *can* share a diffusion model trained on them. The collaborator samples synthetic scans indistinguishable at the pixel level from real ones but containing no actual patient. This is how rare-disease classifiers get enough training data without violating PDPA or HIPAA.

Retail — Virtual Try-On & Product Visualisation

Shopee and Lazada use CycleGAN-style networks (the same adversarial loss family, just with two generators) to paste products onto user photos. A single SKU photo on white background becomes dozens of lifestyle shots — no photo studio, no models, no rental.

Finance — Synthetic Tabular Data for Fraud Models

Banks train fraud models on transaction history, but real fraud examples are rare and sharing them across teams or institutions is blocked by privacy law. Tabular GANs (like CT-GAN, same training dynamics as your WGAN-GP) generate synthetic transaction tables that preserve the joint distribution of fields without containing any real customer. Models trained on the synthetic data generalise to real data at 95%+ of the supervised ceiling.

Creative — Marketing Asset Generation

Agencies use Stable Diffusion for hero images, variant ads, and mood boards. The unit economics are brutal: a hero image that would cost $5,000 in a studio is generated in 30 seconds and iterated 200 times before the art director picks one. Business value is not the pixel quality — it is the *iteration rate*.

Why this matters for YOU: synthetic data is the practical wedge for regulated industries — train models on distributions you cannot legally share. FID (Exercise 5) is the audit evidence: synthetic is close enough to real for the downstream model, without being real.

LESSON 5.6

GRAPH NEURAL NETWORKS

Graph Feature Learning — Patterns in Connected Data

"What if the connections matter as much as the data itself?"

GRAPH DATA: NODES, EDGES, ADJACENCY

COMPONENTS

Nodes (V): entities (people, atoms, papers)

Edges (E): relationships (friendships, bonds, citations)

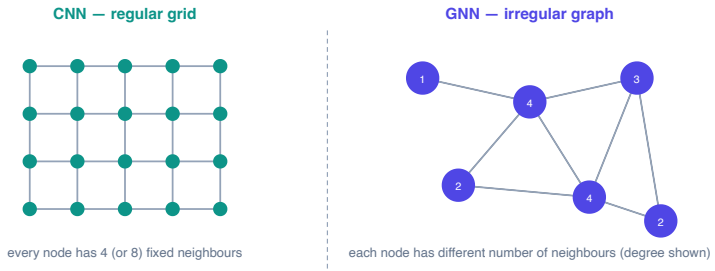
Adjacency matrix (A): $A_{ij} = 1$ if edge between i and j

Node features (X): attributes of each node

APPLICATIONS

Domain	Nodes	Edges
Social	Users	Friendships
Chemistry	Atoms	Bonds
Academic	Papers	Citations
Logistics	Locations	Routes

Why not CNN/RNN? Graphs have variable size, no fixed ordering, irregular connectivity. Standard architectures cannot handle this.



GCN: GRAPH CONVOLUTIONAL NETWORKS

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}H^{(l)}W^{(l)}\right)$$

where $\tilde{A} = A + I$ (add self-loops) and $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$

MESSAGE PASSING (INTUITION)

Aggregate: each node collects features from its neighbours

Transform: apply learnable weight matrix $W^{(l)}$

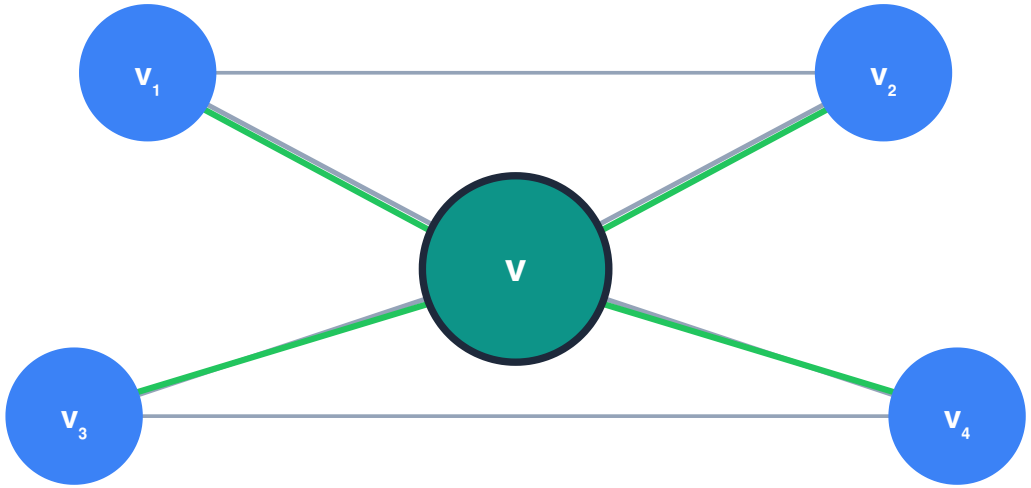
Activate: nonlinear activation (ReLU)

Repeat for L layers — each layer expands the receptive field

Normalisation: $\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}$ is the symmetric normalisation. It prevents nodes with many neighbours from dominating. Equivalent to averaging neighbour features, weighted by degree.

Analogy: Each person updates their opinion by averaging their neighbours' opinions. After a few rounds, everyone's opinion reflects their local community.

GCN: MESSAGE PASSING IN ONE PICTURE



$$h^v \leftarrow \sigma(W \cdot AGG(\{h^u : u \in N(v)\}))$$

center node updates from messages of its neighbours

Green arrows are messages flowing into the centre node. After the update, the centre node's hidden state encodes its 1-hop neighbourhood. Stack L layers and it encodes the L-hop neighbourhood.

THEORY

GRAPHSAGE AND GAT

GRAPHSAGE

Sample a fixed number of neighbours (not all)

Aggregate sampled neighbour features (mean, LSTM, pool)

Inductive: works on unseen nodes (GCN requires full graph)

Advantage: Scales to large graphs. GCN needs the full adjacency matrix in memory. GraphSAGE samples, so it can handle million-node graphs.

GAT (GRAPH ATTENTION NETWORKS)

$$e_{ij} = \text{LeakyReLU}(a^\top [Wh_i \| Wh_j])$$

Attention coefficient for edge (i, j)

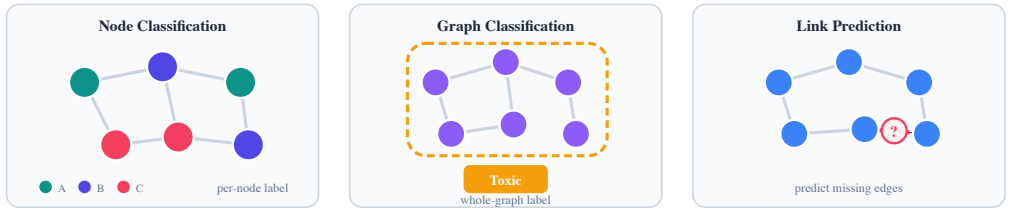
Not all neighbours are equally important

Learn attention weights per edge

Connects to multi-head attention from 5.4

Interpretable: attention weights show important connections

GNN TASK TYPES



Task	Predict	Example
Node	Label per node	User role in social net
Graph	Label per graph	Molecule toxicity
Link	Edge existence	Friend recommendation

GRAPH-LEVEL READOUT

For graph classification, pool node embeddings into one graph vector:

Mean pooling: average all node embeddings

Max pooling: element-wise max

Hierarchical: cluster then pool (DiffPool)

```
from torch_geometric.nn import (
    GCNConv, global_mean_pool
)
# After GCN layers:
x = global_mean_pool(x, batch)
# x is now graph-level embedding
```

EXERCISE 5.6: GRAPH NEURAL NETWORKS

TASKS

- Implement GCN for graph classification on TUDataset
- Compare GCN vs GAT on the same task
- Visualise learned node embeddings (t-SNE)
- Examine GAT attention weights — which edges matter?

ASSESSMENT CRITERIA

- GCN implemented correctly with torch_geometric
- GAT comparison shows attention weight distribution
- Node embeddings visualised and interpreted
- Message passing concept explained in own words

LESSON 5.6 SUMMARY

GNNs learn node representations by aggregating information from neighbours. Message passing is the core mechanism. Attention weights make it interpretable.

KEY EQUATIONS

GCN: $H^{(l+1)} = \sigma(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}H^{(l)}W^{(l)})$

GAT: $e_{ij} = \text{LeakyReLU}(a^\top [Wh_i \| Wh_j])$

WHAT COMES NEXT

Lesson 5.7: **Transfer Learning** — you have now built every major architecture. What if you did not have to train from scratch? Pre-trained models transfer knowledge.

LESSON 5.6: REAL-WORLD APPLICATIONS

Graphs appear wherever the interesting question is "who connects to whom" — molecules, money flows, networks, supply chains. GCN, GAT, and GraphSAGE from Exercise 6 are the off-the-shelf tools for all of them.

Pharma & Biotech — Drug Discovery

Molecules are graphs: atoms are nodes, bonds are edges. GNNs predict solubility, toxicity, and drug-target binding affinity from the graph structure alone — no hand-crafted molecular descriptors. A*STAR's Bioinformatics Institute uses GNN-based virtual screening to narrow 10M-compound libraries down to 1,000 candidates for wet-lab testing.

Telecom — 5G Network Optimisation

Cell towers form a graph with geographic adjacency edges. Singtel and StarHub use GNNs to predict which towers will congest during Formula 1 weekend or the Lunar New Year commute crush — the GAT attention mechanism tells the ops team *which neighbouring tower* is about to offload traffic to the one in question.

Finance — Anti-Money Laundering at MAS-Regulated Banks

Transactions form a graph: accounts are nodes, transfers are edges. AML teams at DBS and OCBC run GNN classifiers over the transaction graph to spot layering and smurfing patterns a rules engine would miss. A single account looks clean; its 3-hop neighbourhood reveals it is the hub of a mule network.

Logistics — PSA Port Cargo Network

Container flows through PSA Singapore are a directed graph: origin port → Singapore → destination port. GNNs predict dwell times and transhipment bottlenecks 48 hours ahead. The GraphSAGE inductive property matters: new ports and new routes can be scored without retraining the full model.

Why this matters for YOU: most analytics treats records as independent. Graph data breaks that — a node's value *comes from* its neighbours. If your business has relationships worth exploiting (customers, accounts, suppliers), GNNs are how you stop discarding that signal.

LESSON 5.7

TRANSFER LEARNING

Knowledge Transfer — Leveraging Pre-Trained Representations

"Why learn from scratch when someone already did the hard part?"

THE TRANSFER LEARNING PARADIGM

CORE IDEA

Pre-train on a large general dataset (ImageNet, BookCorpus)

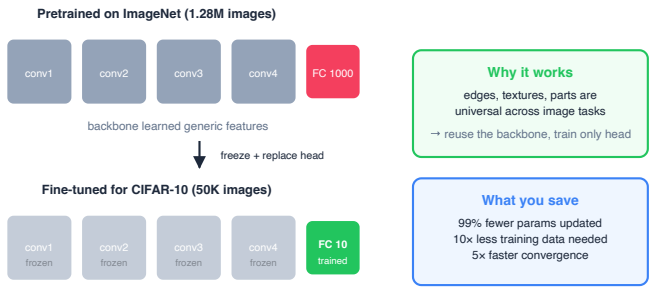
Freeze early layers (they learned universal features)

Fine-tune later layers + new classifier head on your specific task

Works even with very small target datasets (hundreds of examples)

Analogy: Learning to drive a car after already knowing how to ride a bicycle. You do not relearn balance, steering, or traffic rules. You transfer those skills and add car-specific knowledge (gear shifting, mirrors).

Why it works: Early layers learn low-level features (edges, textures for vision; morphology, syntax for text) that are universal. Only the last layers need task-specific adaptation.



CV TRANSFER LEARNING: RESNET FINE-TUNING

STRATEGY

Load pre-trained ResNet (ImageNet weights)

Freeze all layers except the last block

Replace the final FC layer with new classifier head

Fine-tune with low learning rate (1e-4 to 1e-5)

```
import torchvision.models as models

resnet = models.resnet50(pretrained=True)

# Freeze all layers
for param in resnet.parameters():
    param.requires_grad = False

# Replace classifier head
resnet.fc = nn.Linear(2048, num_classes)

# Optionally unfreeze last block
for param in resnet.layer4.parameters():
    param.requires_grad = True
```

DATA AUGMENTATION FOR SMALL DATASETS

Random horizontal flip

Random rotation (up to 15 degrees)

Colour jitter (brightness, contrast)

Random crop with resize

Progressive unfreezing: Start fully frozen. Train head for a few epochs. Then unfreeze the last block. Then the next. This prevents catastrophic forgetting of pre-trained features.

NLP TRANSFER LEARNING: BERT FINE-TUNING

HUGGINGFACE PIPELINE

```
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments, Trainer
)

model_name = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(
    model_name)
model = AutoModelForSequenceClassification \
    .from_pretrained(
        model_name,
        num_labels=num_classes)

args = TrainingArguments(
    learning_rate=2e-5,
    num_train_epochs=3,
    per_device_train_batch_size=16,
    warmup_steps=500)

trainer = Trainer(
    model=model, args=args,
    train_dataset=train_ds,
    eval_dataset=eval_ds)
trainer.train()
```

KEY CONSIDERATIONS

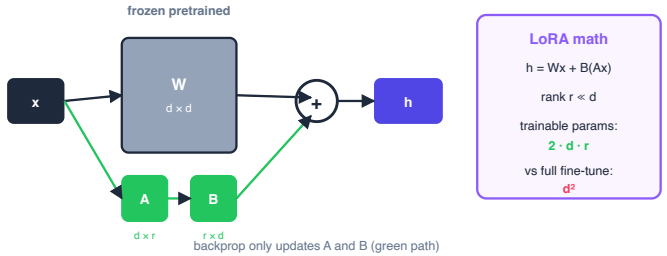
Learning rate: 2e-5 to 5e-5 (BERT paper recommendation)

Epochs: 2-4 (more risks overfitting on small datasets)

Warmup: gradual learning rate increase prevents early instability

Max sequence length: BERT supports up to 512 tokens

Adapter modules: Instead of fine-tuning all parameters, add small bottleneck layers between transformer layers. Train only the adapters. Much fewer parameters to update. Connects to LoRA in M6.



ARCHITECTURE SELECTION GUIDE

Data Type	Best Architecture	When to Transfer	Pre-trained Source
Images	CNN / ViT	Always	ImageNet
Text	Transformer	Always	BERT / GPT
Sequences	LSTM / Transformer	Sometimes	Domain-specific
Graphs	GNN	Rarely	Task-specific
Tabular	Gradient boosting	Never	Train from scratch

Decision rule: If a pre-trained model exists for your data type, USE IT. Transfer learning almost always outperforms training from scratch, especially on small datasets. Only tabular data (the most common in business) resists transfer — gradient boosting still wins there.

KAILASH BRIDGE: ONNXBRIDGE + INFERENCESERVER

FROM FINE-TUNED MODEL TO PRODUCTION

```python
from kailash_ml import (
    OnnxBridge, InferenceServer
)

# Export fine-tuned model
bridge = OnnxBridge()
bridge.export(
    model=fine_tuned_resnet,
    input_shape=(1, 3, 224, 224),
    output_path="mask_detector.onnx"
)

# Serve predictions
server = InferenceServer(
    model_path="mask_detector.onnx"
)
server.warm_cache()

result = server.predict(image_tensor)
print(result.label)      # "mask"
print(result.confidence)  # 0.97
```

INFERENCESERVER API

Method	Purpose
`predict(x)`	Single prediction
`predict_batch(xs)`	Batch predictions
`warm_cache()`	Pre-load model into memory

> **PredictionResult:** Returns `.label`, `.confidence`, `.probabilities`, `.latency_ms`. Structured output, not raw tensors.

EXERCISE 5.7: TRANSFER LEARNING PIPELINE

TASKS

- Fine-tune ResNet for mask detection (CV transfer)
- Fine-tune BERT for text classification (NLP transfer)
- Compare transfer vs training from scratch (both domains)
- Export both models to ONNX with OnnxBridge
- Serve predictions with InferenceServer

ASSESSMENT CRITERIA

- Both models fine-tuned successfully
- Transfer outperforms from-scratch (quantified)
- ONNX export validated
- InferenceServer serves correct predictions
- End-to-end pipeline: train → export → serve

LESSON 5.7 SUMMARY

Transfer learning leverages pre-trained representations. Fine-tune, do not retrain. OnnxBridge exports. InferenceServer deploys.

KEY TAKEAWAYS

Always transfer for images and text
Freeze early layers, fine-tune late layers
Low learning rate (1e-4 to 2e-5)
Progressive unfreezing prevents catastrophic forgetting

WHAT COMES NEXT

Lesson 5.8: **Reinforcement Learning** — all DL so far learns from static data. RL learns from *interaction* with an environment. Actions, rewards, policies.

LESSON 5.7: REAL-WORLD APPLICATIONS

Exercise 7 showed transfer learning beating from-scratch ResNet on CIFAR-10 while using 10× less data. Every application below is built on that same insight: a pre-trained model is the single biggest free lunch in modern ML.

Healthcare — Rare Disease Classification

A public hospital collects 500 labelled scans of a rare condition. From scratch, that is not enough data to train a CNN. Fine-tuning ImageNet-pretrained ResNet or ViT reaches clinically usable accuracy with that exact 500-example budget. This is the only way rare-disease ML exists at all — and it is the exact data-efficiency experiment you ran in Exercise 7.

Manufacturing — Defect Detection with Limited Fault Images

A new product line has no historical defect data, but you need a quality classifier in six weeks. Solution: fine-tune a pre-trained vision backbone on the ~200 defect images you can collect, augmented aggressively. Production-usable accuracy in weeks, not the six months it would take to collect "enough" data.

Finance — Domain-Specific Sentiment on SGX Analyst Reports

Off-the-shelf BERT sentiment trained on Yelp reviews gets the tone of a sell-side report badly wrong ("aggressive growth" is positive in finance, negative elsewhere). A few thousand labelled SGX analyst reports are enough to fine-tune BERT into a finance-specific model — the same NLP transfer pipeline from the exercise.

Retail — Multimodal Product Search on Shopee / Lazada

CLIP-style models pre-trained on billions of (image, caption) pairs, fine-tuned on an e-commerce catalogue, let a user search "minimalist black dining chair" and retrieve the right SKU — without anyone having hand-tagged "minimalist" as an attribute. The pre-training captured the language-vision alignment; the fine-tune made it catalogue-aware.

Why this matters for YOU: transfer learning is the answer to "we don't have enough data" — the most common ML blocker. A pre-trained model plus a few thousand in-domain examples often closes the gap at 5% of the cost and 10% of the timeline.

LESSON 5.8

REINFORCEMENT LEARNING

Learning from Interaction — Policies Learned Through Environment Feedback

"What if the model learned by doing, not by watching?"

THE RL PARADIGM SHIFT

ALL DL SO FAR

Static dataset: images, text, sequences, graphs

Learn patterns from examples

No interaction with the world

Data is given, not generated

REINFORCEMENT LEARNING

Agent interacts with an **environment**

Takes **actions**, receives **rewards**

Learns a **policy** that maximises cumulative reward

Generates its own training data through experience

> **Analogy:** Supervised learning is learning from a textbook. RL is learning by playing a game. Nobody tells you the right move — you try things, see what works, and develop strategy.

RL FUNDAMENTALS

COMPONENTS

Component	Definition
Agent	The learner/decision-maker
Environment	Everything the agent interacts with
State (s)	Current situation
Action (a)	What the agent can do
Reward (r)	Scalar feedback signal
Policy (π)	Mapping from states to actions

THE RL LOOP

Agent observes state s_t

Agent selects action a_t according to policy π

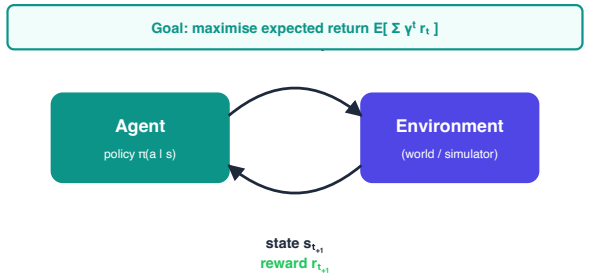
Environment transitions to s_{t+1}

Agent receives reward r_{t+1}

Agent updates policy based on experience

Repeat until episode ends

> **Episode:** A complete sequence from start to termination. The agent aims to maximise *total* reward across the episode, not just immediate reward.



BELLMAN EQUATIONS

VALUE FUNCTION (EXPECTATION)

$$V(s) = \mathbb{E}\left[R_{t+1} + \gamma V(S_{t+1}) \mid S_t = s\right]$$

Expected cumulative reward from state s under current policy

INTUITION

The value of a state = immediate reward + discounted value of the next state. $\gamma \in [0, 1]$ is the discount factor. $\gamma = 0$: greedy. $\gamma = 0.99$: far-sighted.

ACTION-VALUE FUNCTION (OPTIMALITY)

$$Q^*(s, a) = \mathbb{E}\left[R_{t+1} + \gamma \max_{a'} Q^*(S_{t+1}, a') \mid S_t = s, A_t = a\right]$$

Optimal expected cumulative reward for taking action a in state s

KEY DIFFERENCE

$V(s)$ evaluates states. $Q(s, a)$ evaluates state-action pairs. Q^* assumes the agent acts optimally from the next state onward.

DQN: DEEP Q-NETWORK

CORE IDEA

Approximate $Q^*(s, a)$ with a neural network $Q(s, a; \theta)$.

$$\mathcal{L} = \mathbb{E}\left[(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta))^2\right]$$

DQN loss: MSE between prediction and target. θ^- = target network (frozen copy)

KEY INNOVATIONS

Experience replay: store transitions, sample randomly

Target network: frozen copy of Q-network, updated periodically

Epsilon-greedy: explore randomly with probability ϵ

BUSINESS APPLICATION: CUSTOMER CHURN

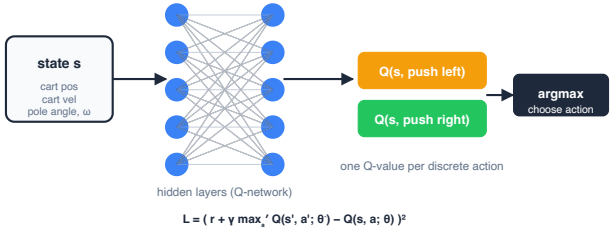
State: customer features (usage, tenure, complaints)

Actions: discount offer, personal call, email campaign, no action

Reward: +1 if customer stays, -cost of action

Policy: which intervention for which customer profile

Why RL, not classification? Churn prediction says "this customer will leave." RL says "do THIS to prevent them from leaving." Action, not just prediction.



POLICY GRADIENT METHODS

VALUE-BASED VS POLICY-BASED

Property	DQN (Value)	Policy Gradient
Learns	Q-values	Policy directly
Actions	Discrete only	Discrete or continuous
Policy	Derived (argmax Q)	Parametrised directly
Exploration	Epsilon-greedy	Stochastic policy

ACTOR-CRITIC ARCHITECTURE

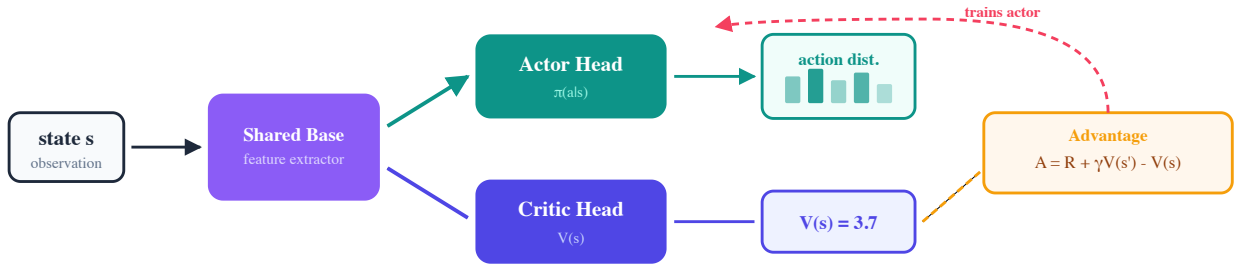
Actor: the policy network $\pi_\theta(a|s)$

Critic: the value network $V_\phi(s)$

Actor proposes actions. Critic evaluates them.

Reduces variance compared to pure policy gradient.

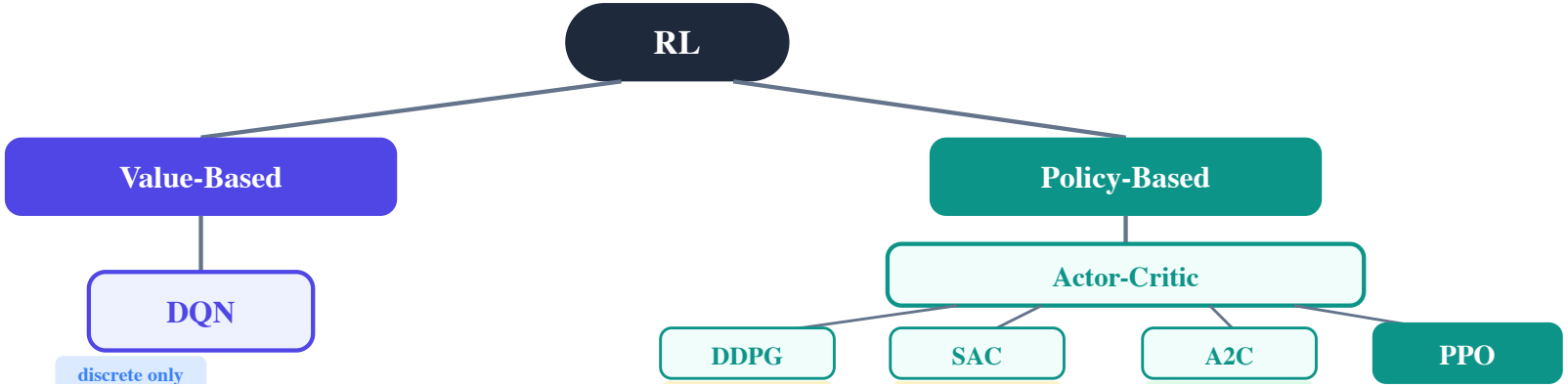
Advantage: $A(s,a) = Q(s,a) - V(s)$. How much better is action a than the average? This centres the gradient estimate and reduces variance.



FIVE ALGORITHMS, FIVE APPLICATIONS

Algorithm	Type	Actions	Business Application
DQN	Value-based	Discrete	Customer churn prevention
DDPG	Actor-Critic	Continuous	Manufacturing control
SAC	Actor-Critic + Entropy	Continuous	Dynamic pricing
A2C	Actor-Critic	Both	Resource allocation
PPO	Actor-Critic (clipped)	Both	Supply chain optimisation

Practical rule: Start with PPO. It is the most stable and works for both discrete and continuous actions. Move to SAC if you need better exploration. Use DQN only for simple discrete problems.



DDPG AND SAC

DDPG: DEEP DETERMINISTIC POLICY GRADIENT

Off-policy actor-critic for **continuous actions**

Deterministic policy: outputs exact action, not distribution

Uses experience replay and target networks (like DQN)

Application: Manufacturing control — continuous adjustments to temperature, pressure, speed

SAC: SOFT ACTOR-CRITIC

Maximum entropy RL: maximise reward AND policy entropy

Entropy bonus: encourages exploration, prevents premature convergence

Automatic temperature tuning

Application: Dynamic pricing — must explore price sensitivity, handle uncertainty

> **Why entropy?** Maximum entropy policies are robust to perturbations. They explore diverse strategies before committing. Critical when the reward landscape is uncertain.

A2C: ADVANTAGE ACTOR-CRITIC

CORE IDEA

Train actor and critic synchronously. Use the **advantage function** to reduce gradient variance.

$$A(s_t, a_t) = r_t + \gamma V(s_{t+1}) - V(s_t)$$

Advantage = actual return - expected baseline

Positive advantage: action was better than expected

Negative advantage: action was worse than expected

Variance reduction via baseline subtraction

APPLICATION: RESOURCE ALLOCATION

State: current resource usage, demand forecast, budget remaining

Actions: allocate resources across departments

Reward: utilisation efficiency, SLA compliance

A2C vs A3C: A3C (Asynchronous) runs multiple workers in parallel. A2C synchronises updates. A2C is simpler and often performs comparably due to GPU batching.

PPO: PROXIMAL POLICY OPTIMIZATION

$$\mathcal{L}^{\text{CLIP}} = \mathbb{E}\left[\min\left(r_t(\theta)A_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t\right)\right]$$

PPO clipped objective — prevents destructively large policy updates

PROBABILITY RATIO

$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ — how much the policy changed. If $r_t = 1$, no change. If $r_t = 2$, action is twice as likely.

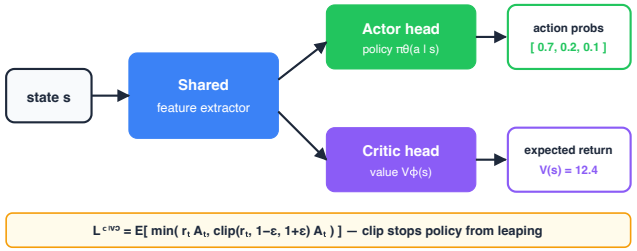
CLIPPING

$\text{clip}(r_t, 1-\epsilon, 1+\epsilon)$ limits the ratio to $[1-\epsilon, 1+\epsilon]$. Typically $\epsilon = 0.2$. This prevents the new policy from straying too far from the old one.

WHY PPO DOMINATES

Simple: no second-order optimisation (unlike TRPO)
Stable: clipping prevents catastrophic updates
Versatile: works for discrete and continuous actions
Scalable: easy to parallelise

RLHF connection: PPO is the algorithm used to align LLMs with human preferences (ChatGPT, Claude). The reward model provides the signal, PPO optimises the policy. More in M6.



CUSTOM GYMNASIUM ENVIRONMENTS

```python
import gymnasium as gym
from gymnasium import spaces
import numpy as np

class ChurnEnv(gym.Env):
    def __init__(self):
        self.action_space = spaces.Discrete(4)
        self.observation_space = spaces.Box(
            low=0, high=1,
            shape=(10,), dtype=np.float32)

    def reset(self, seed=None):
        self.state = self._get_customer()
        return self.state, {}

    def step(self, action):
        reward = self._compute_reward(
            self.state, action)
        self.state = self._next_state(
            self.state, action)
        done = self._is_terminal()
        return self.state, reward, done, \
               False, {}
```

ENVIRONMENT DESIGN CHECKLIST

Define **observation space** (what the agent sees)

Define **action space** (what the agent can do)

Implement **reward function** (what we optimise for)

Implement **transition dynamics** (how actions affect state)

Define **termination conditions** (when episodes end)

Reward shaping matters most: The reward function is the specification of the problem. Sparse rewards (only at episode end) are hard to learn from. Dense rewards (at every step) accelerate learning but can cause reward hacking.

KAILASH BRIDGE: RLTRAINER

RL TRAINING WITH KAILASH

```python
from kailash_ml import RLTrainer

trainer = RLTrainer(
    algorithm="ppo",
    env=ChurnEnv(),
    policy="MlpPolicy",
    learning_rate=3e-4,
    n_steps=2048,
    batch_size=64,
    n_epochs=10,
    gamma=0.99,
    clip_range=0.2
)

# Train the agent
trainer.train(total_timesteps=100_000)

# Evaluate
metrics = trainer.evaluate(
    n_episodes=100
)
print(f"Mean reward: {metrics.mean_reward}")
print(f"Success rate: {metrics.success_rate}")
```

SUPPORTED ALGORITHMS

Algorithm	String
Deep Q-Network	`"dqn"`
DDPG	`"ddpg"`
Soft Actor-Critic	`"sac"`
Advantage Actor-Critic	`"a2c"`
Proximal Policy Opt.	`"ppo"`

> **Pattern:** Define environment → Choose algorithm → Train → Evaluate → Deploy. Same pattern as TrainingPipeline from M4, but for RL.

EXERCISE 5.8: REINFORCEMENT LEARNING

TASKS

- Implement DQN for customer churn prevention (custom environment)
- Implement PPO for supply chain optimisation (custom environment)
- Create custom Gymnasium environments for both
- Compare convergence curves (DQN vs PPO)
- Explain how PPO connects to RLHF (bridge to M6)

ASSESSMENT CRITERIA

- Both algorithms converge (reward increases)
- Custom environments implement correct reward functions
- PPO is more stable than DQN (demonstrated)
- PPO → RLHF connection articulated clearly

> **M6 Preview:** "In M6, the 'environment' is a human evaluator. The 'reward' is human preference. PPO optimises the LLM's policy to align with human values. DPO achieves the same goal without the reward model."

LESSON 5.8 KEY FORMULAS

$$V(s) = \mathbb{E}[R_{t+1} + \gamma V(S_{t+1}) | S_t = s]$$

Bellman expectation equation

$$\mathcal{L}_{\text{DQN}} = \mathbb{E}[(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta))^2]$$

DQN loss

$$Q^*(s, a) = \mathbb{E}[R_{t+1} + \gamma \max_{a'} Q^*(S_{t+1}, a')]$$

Bellman optimality equation

$$\mathcal{L}^{\text{CLIP}} = \mathbb{E}[\min(r_t A_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon) A_t)]$$

PPO clipped objective

LESSON 5.8 SUMMARY

RL learns from interaction, not static data. Bellman equations define value. DQN handles discrete actions. PPO handles everything and connects to RLHF.

KEY TAKEAWAYS

RL = agent + environment + reward + policy
Bellman equations: recursive value definition
DQN: neural network approximates Q-values
PPO: clipped objective prevents catastrophic updates
RLTrainer: train any algorithm with Kailash

WHAT COMES NEXT

Module 6: **LLMs, Alignment, and AI Governance** — PPO becomes RLHF. Transfer learning becomes fine-tuning. Everything you built in M5 is the foundation for production AI systems.

LESSON 5.8: REAL-WORLD APPLICATIONS

Unlike every other lesson, here the exercise environments *are* the applications — you are coding the same decision problems real operations teams face, with the same reward structure.

Telecom — ChurnPrevention (DQN)

Singtel-style retention picks a discount tier per customer per month. Reward = retained revenue – discount cost. DQN learns to save big discounts for customers about to churn, not those who would stay anyway.

Retail — DynamicPricing (SAC)

Lazada / Shopee price 100M SKUs in real time. SAC handles the continuous action (price in a range) and noisy demand — same continuous-action problem as your custom env, at warehouse scale.

Asset Mgmt — PortfolioRebalancing (PPO)

A quant fund rebalances weights under transaction cost and risk. PPO's clipped objective prevents wild reweighting — exactly the clipped-surrogate trick you coded.

Logistics — SupplyChainInventory (PPO)

An FMCG distributor sets daily replenishment for 3,000 SKUs across 40 partners. Reward = service level – holding cost – stockout. RL beats rolling safety-stock by learning real co-movement.

Cloud Ops — ResourceAllocation (A2C)

A platform allocates CPU / memory / I/O across tenants under SLA. A2C's advantage baseline cuts policy-gradient variance — critical when reward is noisy and decisions are high-stakes.

Bridge to M6 — RLHF for LLMs

The PPO you wrote for CartPole is the same algorithm that aligns ChatGPT, Claude, Gemini. Reward = human preference score; action = next token. Exercise 8 is literally the modern LLM alignment loop.

Why this matters for YOU: RL is the right tool for repeated operational decisions under feedback — pricing, inventory, allocation, retention. ROI compounds: a small edge per decision × millions of decisions is material. M6 picks up exactly where this stops.

MODULE 5: COMPLETE FORMULA REFERENCE

Lesson	Formula	Equation
5.1	VAE ELBO	$\mathcal{L} = \mathbb{E}_q[\log p(x\|z)] - D_{\mathrm{KL}}(q(z\|x)\|p(z))$
5.1	Reparameterisation	$z = \mu + \sigma \odot \epsilon$
5.2	Conv output	$O = (W - F + 2P)/S + 1$
5.2	ResNet skip	$\mathcal{H}(x) = \mathcal{F}(x) + x$
5.3	LSTM cell	$C_t = f_t \odot C_{t-1} + i_t \odot \tanh(W_C[h_{t-1}, x_t] + b_C)$
5.4	Attention	$\mathrm{softmax}(QK^\top / \quad d_k)V$
5.4	Positional enc.	$PE_{(pos,2i)} = \sin(pos/10000^{2i/d})$
5.5	GAN minimax	$\min_G \max_D [\mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]]$
5.5	WGAN	$\min_G \max_D [\mathbb{E}[D(x)] - \mathbb{E}[D(G(z))]]$
5.6	GCN	$H^{(l+1)} = \sigma(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}H^{(l)}W^{(l)})$
5.8	Bellman	$V(s) = \mathbb{E}[R + \gamma V(S')]$
5.8	PPO	$\mathbb{E}[\min(r_t A_t, \mathrm{clip}(r_t, 1-\epsilon, 1+\epsilon)A_t)]$

ARCHITECTURE DECISION TREE

WHAT IS YOUR DATA?

Data	Architecture	Lesson
Images	CNN → ViT	5.2
Sequences	LSTM → Transformer	5.3, 5.4
Graphs	GCN / GAT	5.6
Tabular	Gradient boosting (not DL)	M4

WHAT IS YOUR GOAL?

Goal	Architecture	Lesson
Classify	CNN/Transformer + head	5.2, 5.4
Generate	VAE / GAN / Diffusion	5.1, 5.5
Compress	Autoencoder	5.1
Decide	RL (DQN/PPO)	5.8
Transfer	Pre-trained + fine-tune	5.7

Default strategy: Start with a pre-trained model (5.7). If no pre-trained model exists for your domain, choose architecture by data type, train from scratch with the enhancements from this module.

END OF MODULE ASSESSMENT

QUIZ COMPONENT

Formula derivations (VAE ELBO, attention scaling, LSTM gates)

Architecture selection (given a problem, choose and justify)

Concept explanations (reparameterisation trick, skip connections, mode collapse)

DL ARCHITECTURE PROJECT

Choose a problem (image, text, graph, or RL)

Select and justify the architecture

Train the model with appropriate enhancements

Evaluate with proper metrics

Export to ONNX and deploy with InferenceServer

> **Full pipeline:** Problem → Architecture → Train → Evaluate → Export → Serve. End to end.

MODULE 5 COMPLETE

DEEP LEARNING AND MACHINE LEARNING MASTERY IN VISION AND TRANSFER LEARNING

You can now build, train, and deploy every major DL architecture — from autoencoders to RL agents.

"The architectures are tools. The judgement of which tool to use — that is mastery."

MLFP — ML Foundations for Professionals — Terrene Open Academy

Powered by Kailash Python SDK — Terrene Foundation